

华中科技大学

深度学习实践
结课报告

2026

专 业:	光学与电子信息
班 级:	光电 2211
学 号:	U202211671
姓 名:	陈增
电 话:	13365760706
邮 件:	2894726086@qq.com
完成日期:	2026. 5

实验报告及电路设计评分细则

评 分 项 目	满分	得分	备注	
文档格式（段落、行间距、缩进、图表、编号等）	10		实验报告 总分	
任务一	15			
任务二	20			
任务三	25			
任务四	20			
遇到的问题及建议	10			
教师签名			日 期	

备注：实验过程将从关键代码的完整性、中间结果、训练结果、实践训练程度等方面进行评分。

任务一 识别与分类

1.1. 任务说明

任务一基于 MNIST 手写数字数据集（采用 Kaggle 比赛图像目录结构格式进行重构与组织）的手写数字识别分类任务，具体分为三个子任务，分别构建三种神经网络模型（FCN、CNN、Transformer）的图像分类系统并比较区别。

任务涵盖底层跨平台环境配置（Anaconda+python+pyTorch）、具备有标签/无标签双模式解耦的通用数据加载器（Dataset 与 DataLoader）设计、神经网络搭建、超参数自适应收敛训练以及面向 Kaggle 测试集的未标注单样本大规模批量推理，探索在非线性映射下隐层特征的高效提取表达。

这里的 MNIST 数据集（MNIST_kaggle）不是标准版，区别于传统标准的 MNIST（60000 张训练图，10000 张测试图），对于标准版的 MNIST，可以直接使用 `torchvision.datasets.MNIST` 来加载和处理数据，但这里我们需要自己实现数据加载逻辑。这里的数据集是 kaggle 上提供的一个变体（比赛专用版），包含训练集和测试集两部分，且测试集（28000 张）没有标签。我们需要训练模型在训练集上，并在测试集上做预测，提交预测结果到 kaggle 官网进行评测。

环境配置方面，本次实践使用 python 语言，为了方便可视化交互与代码调试，在 VSCode 中采用 Jupyter Notebook 形式编写 ipynb 文件。使用 anaconda 进行 python 环境管理，创建了虚拟环境 DL(python3.9.25)，其中安装了 pytorch、numpy、matplotlib 等需要的库。

1.2 子任务一：实现一个简单全连接网络 FCN，训练并评估对 MNIST_kaggle 数据集的分类效果

1.2.1 FCN 介绍

FCN（Fully Connected Network，全连接网络），在深度学习中通常也被称为 MLP（Multi-Layer Perceptron，多层感知机），是最基础、最经典的神经网络架构之一。

它的核心特点是：上一层的每一个神经元，都与下一层的每一个神经元进行连接。

- 单个神经元

神经元是神经网络的基本单元，它接收输入信号，通过加权求和后与偏置（bias）相加，然后通过激活函数处理以产生输出。

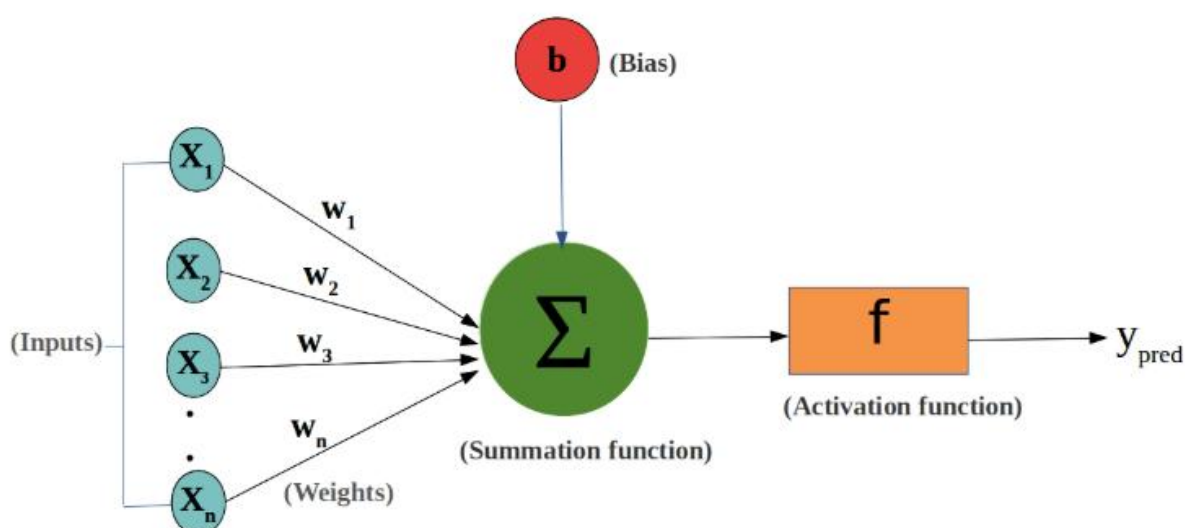
神经元的权重和偏置是网络学习过程中需要调整的参数。

输入和输出：

输入（Input）：输入是网络的起始点，可以是特征数据，如图像的像素值或文本的词向量。

输出（Output）：输出是网络的终点，表示模型的预测结果，如分类任务中的类别标签。

神经元接收多个输入（例如 $x_1, x_2, \dots, x_n$ ），将输入线性组合的结果传入激活函数进行非线性激活后输出。如图所示：



神经元的输出 z 可以看作是输入的加权和加上偏置（bias），然后传入激活函数，数学表示为：

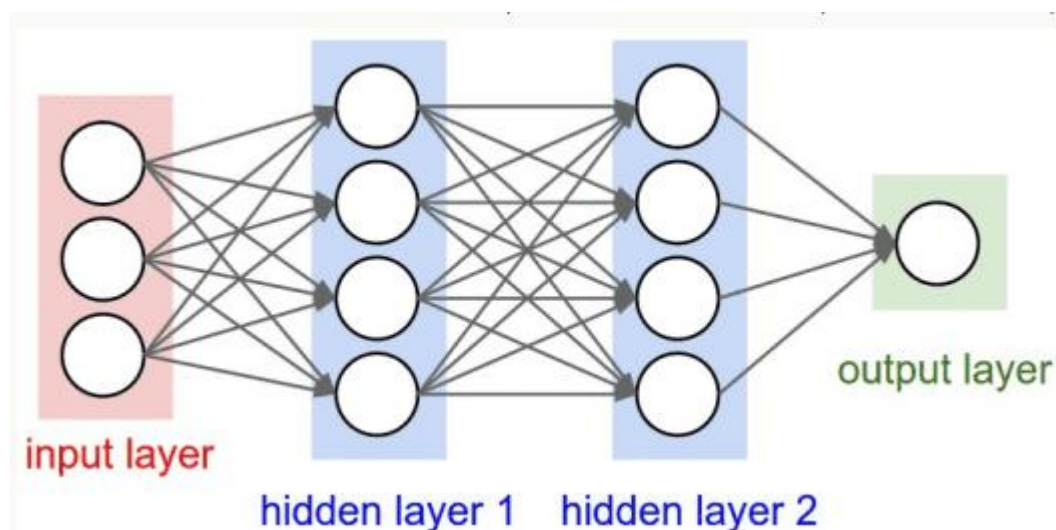
$$z = \sum_j w_j x_j + \text{Bias}$$

$$y_{\text{pred}} = f\left(\sum_j w_j x_j + \text{Bias}\right)$$

这里， w_j 是权重， x_j 是输入，而 Bias 是偏置项。

- FCN 的基本结构组成

一个标准的全连接网络通常由以下三层按顺序串联组成：



1. 输入层 (Input Layer)

功能：负责接收原始数据。

特点：输入层本身不进行任何数学计算。在处理图像等高维数据时，需要先将多维数据（如 28×28 的二维图像）“展平”（Flatten）为一维向量（784 维），才能送入输入层。

2. 隐藏层 (Hidden Layer)

功能：负责从输入数据中提取特征。一个网络可以包含单个或多个隐藏层（即“深度”）。

其中每个神经元执行两个步骤：

线性组合：将上一层所有输入乘以各自的权重（Weights）并求和，再加上偏置（Bias）：

$$z = \sum (w_i \cdot x_i) + b$$

非线性激活：将线性结果 z 传入激活函数（如 ReLU、Sigmoid、Tanh），引入非线性表达能力，使网络能够拟合复杂的非线性边界。

3. 输出层 (Output Layer)

功能：产生最终的预测结果。

设计规则：其神经元数量由具体任务决定：

二分类任务：通常为 1 个神经元（配合 Sigmoid 输出概率）。

多分类任务（如手写数字 0-9 识别）：通常为 10 个神经元（每个神经元代表一个类别的得分，配合 Softmax 转化为概率分布）。

1.2.2 关键代码及说明

(1) 安装与启动 Jupyter 内核 （VS Code）

```
1) 安装与启动 Jupyter 内核 （VS Code）

首次使用可在此单元安装依赖；运行后在 VS Code 右上角选择 Python 内核（选择配置好的 DL 环境：python 3.9.25）。

# 如已安装可跳过
# %pip install -U pip
# %pip install ipykernel notebook torch torchvision pillow

import sys
print(sys.executable)
print("内核已就绪")

[3] ✓ 0.0s Python
... d:\anaconda\envs\DL\python.exe
内核已就绪
```

(2) 导入依赖并检查 PyTorch 版本

```
2) 导入依赖并检查 PyTorch 版本

导入训练与数据处理所需模块，并确认当前 PyTorch 版本。

import os
import re
import random
import torch
import pandas as pd
import torch.nn as nn
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader

print("torch version:", torch.__version__)

[5] ✓ 2.6s Python
... torch version: 2.8.0+cpu
```

(3) 配置数据路径（基于当前工作目录）

```
3) 配置数据路径（基于当前工作目录）

project_root = os.path.dirname(os.getcwd()) # 当前工作目录

train_dir = os.path.join(project_root, "MNIST_kaggle", "trainingSet")
test_dir = os.path.join(project_root, "MNIST_kaggle", "testSet")

print("cwd:", os.getcwd())
print("project_root:", project_root)
print("train_dir exists:", os.path.exists(train_dir), "train_dir:", train_dir)
print("test_dir exists:", os.path.exists(test_dir), "test_dir:", test_dir)

[6] ✓ 0.0s Python
... cwd: d:\CZ_WorkSpace\ai学习\深度学习实践\task1\task1_1
project_root: d:\CZ_WorkSpace\ai学习\深度学习实践\task1
train_dir exists: True train_dir: d:\CZ_WorkSpace\ai学习\深度学习实践\task1\MNIST_kaggle\trainingSet
test_dir exists: True test_dir: d:\CZ_WorkSpace\ai学习\深度学习实践\task1\MNIST_kaggle\testSet
```

(4) 定义 MyDataset（有标签/无标签双模式）

这里自定义了 MyDataset：通过继承 torch.utils.data.Dataset 来加载自己的数据集。目标是允许自己的数据源中创建数据集。

继承该类的关键在于

__len__(self)：返回数据集中的样本数量。

__getitem__(self, idx)：如何通过索引返回一个样本。

这里的 MyDataset 适配了 kaggle 数据集的文件夹，支持训练集有标签和测试集无标签（使用 labeled = True/False 区分）。

4) 定义 MyDataset（有标签/无标签双模式）

保留原有数据集逻辑：灰度化、缩放、ToTensor、展平 784；支持训练集有标签和测试集无标签。

```
class MyDataset(Dataset):
    def __init__(self, data_dir, labeled=True):
        self.data_dir = data_dir
        self.labeled = labeled
        self.samples = []
        self.transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((28, 28)),
            transforms.ToTensor(),
        ])

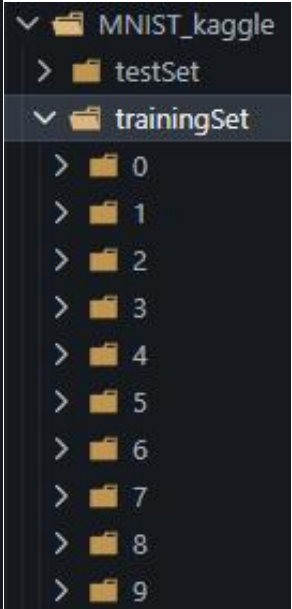
    if self.labeled:
        class_names = [d for d in os.listdir(data_dir) if os.path.isdir(os.path.join(data_dir, d))]
        class_names = sorted(class_names, key=lambda x: int(x) if x.isdigit() else x)
        self.class_to_idx = {name: int(name) for name in class_names}
        for class_name in class_names:
            class_dir = os.path.join(data_dir, class_name)
            for file_name in os.listdir(class_dir):
                if file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                    file_path = os.path.join(class_dir, file_name)
                    label = self.class_to_idx[class_name]
                    self.samples.append((file_path, label))
    else:
        self.class_to_idx = {}
        for file_name in os.listdir(data_dir):
            file_path = os.path.join(data_dir, file_name)
            if os.path.isfile(file_path) and file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                self.samples.append(file_path)

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        if self.labeled:
            file_path, label = self.samples[idx]
        else:
            file_path = self.samples[idx]
            label = None

        image = Image.open(file_path)
        image = self.transform(image)
        image = image.view(-1) # [1, 28, 28] -> [784]

        if self.labeled:
            return image, torch.tensor(label, dtype=torch.long)
        return image, file_path
```



图片处理过程中用到了 PyTorch 提供的 torchvision.transforms 模块来进行常见的图像预处理和增强操作，如旋转、裁剪、归一化等。

transforms.Compose()：将多个变换操作组合在一起。

transforms.Resize()：调整图像大小。

`transforms.ToTensor()`: 将图像转换为 PyTorch 张量, 值会被归一化到 $[0, 1]$ 范围。

`transforms.Normalize()`: 标准化图像数据, 通常使用预训练模型时需要进行标准化处理。

(5) 构建 DataLoader 并检查 batch 形状

`DataLoader` 是 PyTorch 提供的一个重要工具, 用于从 `Dataset` 中按批次 (batch) 加载数据。

`DataLoader` 允许我们批量读取数据并进行多线程加载, 从而提高训练效率。重要属性有:

`batch_size`: 每次加载的样本数量。

`shuffle`: 是否对数据进行洗牌, 通常训练时需要将数据打乱。

`drop_last`: 如果数据集中的样本数不能被 `batch_size` 整除, 设置为 `True` 时, 丢弃最后一个不完整的 batch。

5) 构建 DataLoader 并检查 batch 形状

训练集使用 `labeled=True`, 测试集按当前结构使用 `labeled=False`。

```
mydataset = MyDataset(data_dir=train_dir, labeled=True)
mydataset_loader = DataLoader(mydataset, batch_size=32, shuffle=True)

test_dataset = MyDataset(data_dir=test_dir, labeled=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print(f"Train samples: {len(mydataset)}")
print(f"Test samples: {len(test_dataset)}")

for batch_idx, (images, labels) in enumerate(mydataset_loader):
    print(f"Batch {batch_idx}: images={images.shape}, labels={labels.shape}")
    print("Label sample:", labels[:10].tolist())
    if batch_idx == 1:
        break
```

[8] ✓ 1.3s

```
... Train samples: 42000
Test samples: 28000
Batch 0: images=torch.Size([32, 784]), labels=torch.Size([32])
Label sample: [9, 4, 1, 2, 0, 9, 1, 2, 3, 8]
Batch 1: images=torch.Size([32, 784]), labels=torch.Size([32])
Label sample: [2, 2, 6, 2, 0, 4, 5, 1, 2, 9]
```

整个任务一 (1-1, 1-2, 1-3) 都是使用同样的数据集, 所以后面两个子任务构建和加载数据集的方法和 1-1 的相同, 故后面不再介绍细节。

(6) 定义全连接网络 MyNetwork

在 PyTorch 中，构建神经网络通常需要继承 `nn.Module` 类。

`nn.Module` 是所有神经网络模块的基类，你需要定义以下两个部分：

- `__init__()`：定义网络层。
- `forward()`：定义数据的前向传播过程。

PyTorch 提供了许多常见的神经网络层，以下是几个常见的：

- `nn.Linear(in_features, out_features)`：全连接层，输入 `in_features` 个特征，输出 `out_features` 个特征。
- `nn.Conv2d(in_channels, out_channels, kernel_size)`：2D 卷积层，用于图像处理。
- `nn.MaxPool2d(kernel_size)`：2D 最大池化层，用于降维。
- `nn.ReLU()`：ReLU 激活函数，常用于隐藏层。
- `nn.Softmax(dim)`：Softmax 激活函数，通常用于输出层，适用于多类分类问题。

```
6) 定义全连接网络 MyNetwork

网络结构为 784 -> 128 -> 10，输出 logits。

class MyNetwork(nn.Module):
    def __init__(self):
        super(MyNetwork, self).__init__()
        self.layer1 = nn.Linear(784, 128)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x
```

我们这里的 FCN 结构为 784-128-10，激活函数选用的是 ReLU，最后输出的结果是 logits（逻辑值）。

逻辑值是实数向量（在这里是 10 维），可正可负，没有范围限制，某个维度的越大代表当前维度代表的分类概率越大。逻辑值需要经过 softmax 之后才是概率值。

在训练计算损失函数（Cross Entropy Loss）时，直接使用 Logits 进行计算在数学上更稳定，能避免计算机出现“下溢出”

(7) 定义评估函数 evaluate 与单样本推理 single_sample_check

7) 定义评估函数 evaluate 与单样本推理 single_sample_check

测试集无标签时会跳过准确率；单样本推理输出预测类别与置信度。

```
def evaluate(model, data_loader):
    if len(data_loader.dataset) == 0 or not data_loader.dataset.labeled:
        print("Warning: test set has no labels, skip accuracy evaluation.")
        return None

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in data_loader:
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

    model.train()
    if total == 0:
        print("Warning: test set is empty, skip accuracy evaluation.")
        return None
    return correct / total
```

```
def single_sample_check(model, dataset):
    if len(dataset) == 0:
        print("Single image validation skipped: test dataset is empty.")
        return

    sample_idx = random.randint(0, len(dataset) - 1)
    sample = dataset[sample_idx]
    if dataset.labeled:
        image, label = sample
        image_path, _ = dataset.samples[sample_idx]
    else:
        image, image_path = sample
        label = None

    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0))
        probs = torch.softmax(logits, dim=1)
        pred = torch.argmax(probs, dim=1).item()
        confidence = probs[0, pred].item()
    model.train()

    print("\nSingle Image Validation")
    print(f"Image: {image_path}")
    if label is not None:
        print(f"True Label: {label.item()}, Pred Label: {pred}, Confidence: {confidence:.4f}")
    else:
        print(f"Pred Label: {pred}, Confidence: {confidence:.4f} (no ground-truth label)")
```

评估函数主要是用于评估在每个 epoch 结束后计算对测试集分类的准确率（预测正确的/总样本数），从而查看在测试集分类的结果随着 epoch 训练轮次的变化情况。

但是在本实验中因为测试集没有标签，所以无法计算准确率，但是考虑到完整性仍然保留这个函数。

单样本推理 single_sample_check 主要是使用某一个时刻的模型对单个样本进行推理，输出置信结果。

(8) 定义损失函数和优化器

8) 定义损失函数与优化器

使用 CrossEntropyLoss 与 Adam(lr=0.001)，并做一次随机输入可用性检查。

```
loss_fn = nn.CrossEntropyLoss()
out = MyNetwork()
optimizer = torch.optim.Adam(out.parameters(), lr=0.001)

# 快速可用性检查
_ = loss_fn(out(torch.randn(32, 784)), torch.randint(0, 10, (32,)))
print("model check ok")
```

[11] ✓ 0.0s

... model check ok

(9) 编写训练循环（含测试评估与模型保存）

9) 编写训练循环（含测试评估与模型保存）

每个 epoch 执行训练；若测试集有标签则评估准确率；每 2 个 epoch 保存一次模型。

```
> epochs = 10
os.makedirs("checkpoints", exist_ok=True)

epoch_losses = []
epoch_accs = []

for epoch in range(epochs):
    out.train()
    running_loss = 0.0
    sample_count = 0
    for batch_idx, (images, labels) in enumerate(mydataset_loader):
        output = out(images)
        loss = loss_fn(output, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        batch_size = labels.size(0)
        running_loss += loss.item() * batch_size
        sample_count += batch_size

        print(f"Epoch {epoch}, Batch {batch_idx}, Loss: {loss.item():.6f}")

    epoch_loss = running_loss / max(sample_count, 1)
    epoch_losses.append(epoch_loss)

    test_acc = evaluate(out, test_loader)
    if test_acc is not None:
        epoch_accs.append(test_acc)
        print(f"Epoch {epoch}, Test Accuracy: {test_acc * 100:.2f}%")
    else:
        epoch_accs.append(None)

    if epoch % 1 == 0:
        ckpt_path = os.path.join("checkpoints", f"model_epoch_{epoch}.pth")
        torch.save(out.state_dict(), ckpt_path)
        print("Saved:", ckpt_path)
```

[12] ✓ 2m 22.0s

Epoch 9, Batch 1285, Loss: 0.045216
Epoch 9, Batch 1286, Loss: 0.006535
Epoch 9, Batch 1287, Loss: 0.002039
Epoch 9, Batch 1288, Loss: 0.001808
Epoch 9, Batch 1289, Loss: 0.023697
Epoch 9, Batch 1290, Loss: 0.004224
Epoch 9, Batch 1291, Loss: 0.111605
Epoch 9, Batch 1292, Loss: 0.007341
Epoch 9, Batch 1293, Loss: 0.036595
Epoch 9, Batch 1294, Loss: 0.004017
Epoch 9, Batch 1295, Loss: 0.004297
Epoch 9, Batch 1296, Loss: 0.010994
Epoch 9, Batch 1297, Loss: 0.006246
Epoch 9, Batch 1298, Loss: 0.014269
Epoch 9, Batch 1299, Loss: 0.028469
Epoch 9, Batch 1300, Loss: 0.001744
Epoch 9, Batch 1301, Loss: 0.000777
Epoch 9, Batch 1302, Loss: 0.015618
Epoch 9, Batch 1303, Loss: 0.014744
Epoch 9, Batch 1304, Loss: 0.026320
Epoch 9, Batch 1305, Loss: 0.049100
Epoch 9, Batch 1306, Loss: 0.132014
Epoch 9, Batch 1307, Loss: 0.014700
Epoch 9, Batch 1308, Loss: 0.052309
Epoch 9, Batch 1309, Loss: 0.001889
Epoch 9, Batch 1310, Loss: 0.007933
Epoch 9, Batch 1311, Loss: 0.009830
Epoch 9, Batch 1312, Loss: 0.006693
Warning: test set has no labels, skip accuracy evaluation.
Saved: checkpoints\model_epoch_9.pth

```
# 9.1) 训练曲线可视化
epochs_axis = list(range(1, len(epoch_losses) + 1))

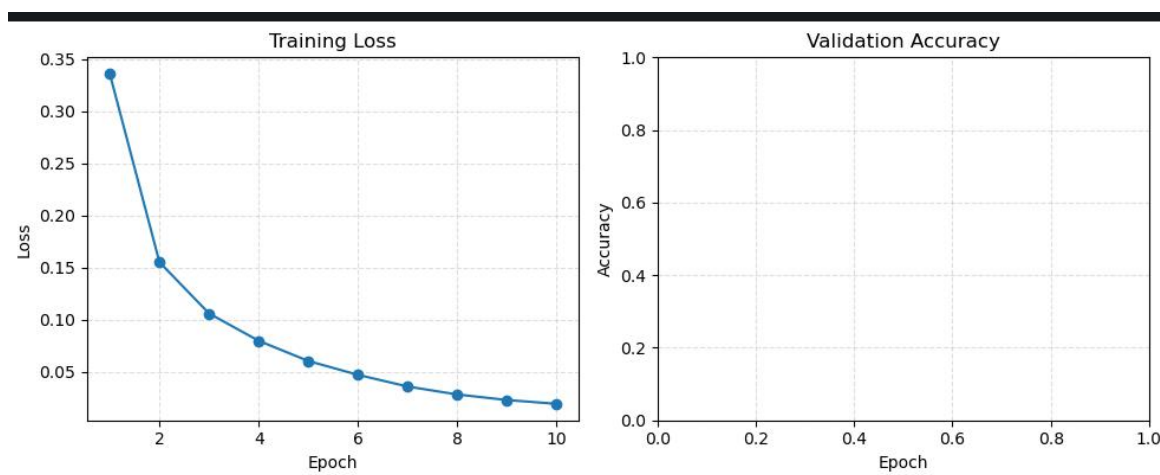
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_axis, epoch_losses, marker="o")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True, linestyle="--", alpha=0.4)

plt.subplot(1, 2, 2)
valid_acc = [a for i, a in enumerate(epoch_accs) if a is not None]
valid_epochs = [i + 1 for i, a in enumerate(epoch_accs) if a is not None]
if valid_acc:
    plt.plot(valid_epochs, valid_acc, marker="o", color="green")
    plt.ylim(0, 1)
plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True, linestyle="--", alpha=0.4)

plt.tight_layout()
plt.show()
```

✓ 0.2s

编写训练循环并进行 Loss 与 Accuracy 可视化，由于无法计算测试集的 Accuracy，故 Accuracy 的可视化无结果。



(10) 训练后单图验证与可选模型加载推理

10) 训练后单图验证与可选模型加载推理

先做一次单图验证；再演示如何加载保存的权重做推理。

```
# 训练后单图验证
single_sample_check(out, test_dataset)

# 可选：加载某个 checkpoint 后再做单图推理
# model2 = MyNetwork()
# model2.load_state_dict(torch.load(os.path.join("checkpoints", "model_epoch_8.pth"), map_location="cpu"))
# single_sample_check(model2, test_dataset)
```

Single Image Validation
Image: d:\CZ_WorkSpace\ai学习\深度学习实践\task1\MNIST_kaggle\testSet\img_332.jpg
Pred Label: 3, Confidence: 0.9998 (no ground-truth label)

(11) 加载模型参数并对测试集推理（导出 Kaggle 的 csv 格式提交文件）

```
# 11) 加载模型参数并对测试集推理（导出 Kaggle 提交文件）
import pandas as pd
from torch.utils.data import DataLoader

# TODO: 修改为你要加载的权重文件路径
ckpt_path = os.path.join("checkpoints", "model_epoch_8.pth")

infer_model = MyNetwork()
infer_model.load_state_dict(torch.load(ckpt_path, map_location="cpu"))
infer_model.eval()

# 测试集无标签，推理时 batch 处理
infer_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

pred_records = []
with torch.no_grad():
    for images, file_paths in infer_loader:
        logits = infer_model(images)
        preds = torch.argmax(logits, dim=1).cpu().numpy().tolist()
        for path, pred in zip(file_paths, preds):
            base = os.path.basename(path)
            match = re.search(r"d+", base)
            if match is None:
                raise ValueError(f"Cannot parse image id from filename: {base}")
            image_id = int(match.group())
            pred_records.append((image_id, pred))

# 按 ImageId 数值升序，符合 Kaggle 提交要求
pred_records.sort(key=lambda x: x[0])
submit_df = pd.DataFrame(pred_records, columns=["ImageId", "Label"])

# 保存为 CSV，方便提交到 Kaggle
submit_path = "kaggle_submit_FCN.csv"
submit_df.to_csv(submit_path, index=False)
print(f"Saved submission file: {submit_path}")
print(submit_df.head())
```

Saved submission file: kaggle_submit_FCN.csv

ImageId	Label
0	1
1	2
2	3
3	4

task1

MNIST_kaggle

task1_1

checkpoints

model_epoch_0.pth

model_epoch_1.pth

model_epoch_2.pth

model_epoch_3.pth

model_epoch_4.pth

model_epoch_5.pth

model_epoch_6.pth

model_epoch_7.pth

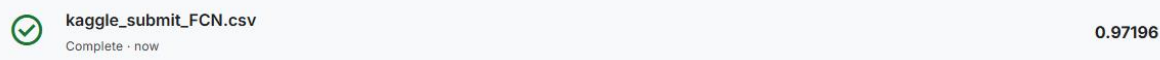
model_epoch_8.pth

model_epoch_9.pth

kaggle_submit_FCN.csv

task1_1.ipynb

在 kaggle 官网提交测试，得到准确率为 0.97196，说明成功完成了分类任务



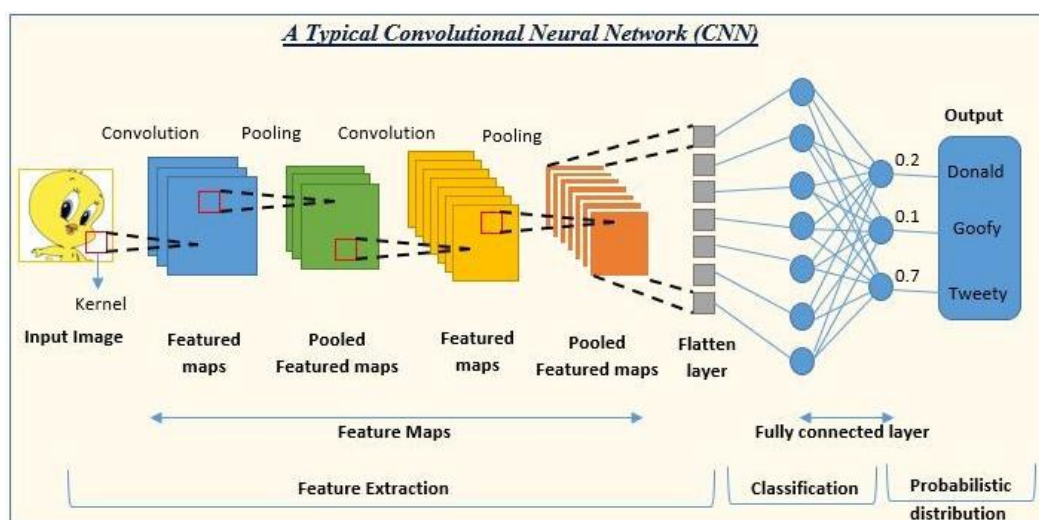
1.3 子任务二：实现卷积神经网络 (CNN)，训练并评估 MNIST_kaggle 数据集的分类效果

子任务二引入了卷积神经网络 (CNN) 来替换子任务一中的全连接网络。CNN 通过“局部连接”和“权重共享”两大核心特性，能够有效提取图像的空间结构特征（如边缘、纹理和轮廓等），同时大幅度减少网络的参数量，从而解决 FCN 在处理图像任务时因破坏空间邻近关系且参数量过大而易导致过拟合的问题。

1.3.1 CNN 介绍

卷积神经网络 (Convolutional Neural Networks, CNN) 是一类专门用于处理具有网格状拓扑结构数据（如图像）的深度学习模型。是计算机视觉任务（如图像分类、目标检测和分割）的核心技术。

下面这张图展示了一个典型的卷积神经网络 (CNN) 的结构和工作流程，用于图像识别任务。



在图中，CNN 的输出层给出了三个类别的概率：Donald (0.2)、Goofy (0.1) 和 Tweety (0.7)，这表明网络认为输入图像最有可能是 Tweety。

- 卷积神经网络的基本结构

- 1、输入层 (Input Layer)

接收原始图像数据，图像通常被表示为一个三维数组，其中两个维度代表图像的宽度和高度，第三个维度代表颜色通道（例如，RGB 图像有三个通道）。

2、卷积层（Convolutional Layer）

用卷积核提取局部特征，如边缘、纹理等。

公式：

$$y[i,j] = \sum_m \sum_n x[i+m,j+n] \cdot k[m,n] + b$$

x: 输入图像。

k: 卷积核（权重矩阵）。

b: 偏置。

应用一组可学习的滤波器（或卷积核）在输入图像上进行卷积操作，以提取局部特征。

每个滤波器在输入图像上滑动，生成一个特征图（Feature Map），表示滤波器在不同位置的激活。

卷积层可以有多个滤波器，每个滤波器生成一个特征图，所有特征图组成一个特征图集合。

3、激活函数（Activation Function）

通常在卷积层之后应用非线性激活函数，如 ReLU（Rectified Linear Unit），以引入非线性特性，使网络能够学习更复杂的模式。

ReLU 函数定义为： $f(x) = \max(0, x)$ ，即如果输入小于 0 则输出 0，否则输出输入值。

4、池化层（Pooling Layer）

用于降低特征图的空间维度，减少计算量和参数数量，同时保留最重要的特征信息。

最常见的池化操作是最大池化（Max Pooling）和平均池化（Average Pooling）。

最大池化选择区域内的最大值，而平均池化计算区域内的平均值。

5、归一化层（Normalization Layer，可选）

例如，局部响应归一化（Local Response Normalization, LRN）或批归一化（Batch Normalization）。

这些层有助于加速训练过程，提高模型的稳定性。

6、全连接层（Fully Connected Layer）

在 CNN 的末端，将前面层提取的特征图展平（Flatten）成一维向量，然后输入到全连接层。

全连接层的每个神经元都与前一层的所有神经元相连，用于综合特征并进行最终的分类或回归。

7、输出层（Output Layer）

根据任务的不同，输出层可以有不同的形式。

对于分类任务，通常使用 Softmax 函数将输出转换为概率分布，表示输入属于各个类别的概率。

8、损失函数（Loss Function）

用于衡量模型预测与真实标签之间的差异。

常见的损失函数包括交叉熵损失（Cross-Entropy Loss）用于多分类任务，均方误差（Mean Squared Error, MSE）用于回归任务。

9、优化器（Optimizer）

用于根据损失函数的梯度更新网络的权重。常见的优化器包括随机梯度下降（SGD）、Adam、RMSprop 等。

10、正则化（Regularization，可选）

包括 Dropout、L1/L2 正则化等技术，用于防止模型过拟合。

这些层可以堆叠形成更深的网络结构，以提高模型的学习能力。

CNN 的深度和复杂性可以根据任务的需求进行调整。

1.3.2 关键代码及说明

子任务二的流程与子任务一基本相同，主要区别在于

(1) 数据集的构建不同

(2) 神经网络构建不同

接下来主要展示与子任务一不同的地方的代码细节和训练过程及结果

(1) 数据集的构建不同

与任务一的输出展平为 784 维输入不同，CNN 的输入为 28*28*1 的形式。

4) 定义 MyDataset (有标签/无标签双模式)

CNN 输入为 [1, 28, 28], 这里不再展平成 784。

```
class MyDataset(Dataset):
    # 初始化数据集, 支持有标签/无标签两种模式
    def __init__(self, data_dir, labeled=True):
        # 保存数据目录与标签标记
        self.data_dir = data_dir
        self.labeled = labeled
        # samples 保存 (路径, 标签) 或 路径
        self.samples = []
        # 定义统一的图像预处理流程
        self.transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((28, 28)),
            transforms.ToTensor(),
        ])

    # 如果是训练集, 读取子目录作为类别
    if self.labeled:
        # 获取所有类别目录名
        class_names = [d for d in os.listdir(data_dir) if os.path.isdir(os.path.join(data_dir, d))]
        # 按数字顺序排序类别目录
        class_names = sorted(class_names, key=lambda x: int(x) if x.isdigit() else x)
        # 构建类别名到整数标签的映射
        self.class_to_idx = {name: int(name) for name in class_names}
        # 遍历每个类别目录收集样本
        for class_name in class_names:
            class_dir = os.path.join(data_dir, class_name)
            for file_name in os.listdir(class_dir):
                # 只保留图片文件
                if file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                    file_path = os.path.join(class_dir, file_name)
                    label = self.class_to_idx[class_name]
                    self.samples.append((file_path, label))
        else:
            # 测试集无标签, 仅保存图片路径
            self.class_to_idx = {}
            for file_name in os.listdir(data_dir):
                file_path = os.path.join(data_dir, file_name)
                # 只保留图片文件
                if os.path.isfile(file_path) and file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                    self.samples.append(file_path)

    # 返回样本数量
    def __len__(self):
        return len(self.samples)

    # 获取单个样本
    def __getitem__(self, idx):
        # 根据是否有标签拆包
        if self.labeled:
            file_path, label = self.samples[idx]
        else:
            file_path = self.samples[idx]
            label = None

        # 读取图片并做预处理
        image = Image.open(file_path)
        image = self.transform(image)

        # 训练集返回 (图像张量, 标签)
        if self.labeled:
            return image, torch.tensor(label, dtype=torch.long)
        # 测试集返回 (图像张量, 文件路径)
        return image, file_path
```

(2) 神经网络构建不同

6) 定义卷积神经网络 MyCNN

结构: Conv(1->16) -> ReLU -> MaxPool -> Conv(16->32) -> ReLU -> MaxPool -> FC。

```
class MyCNN(nn.Module):
    # 初始化 CNN 网络结构
    def __init__(self):
        super(MyCNN, self).__init__()
        # 卷积特征提取部分
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        # 分类器: 展平后接全连接层输出 logits
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10),
        )

    # 前向传播逻辑
    def forward(self, x):
        # 提取卷积特征
        x = self.features(x)
        # 通过分类器得到最终输出
        x = self.classifier(x)
        return x
```

✓ 0.0s

Python

训练过程及结果

9) 训练循环 (含评估与模型保存)

模型和提交文件保存到 task1_2 目录下。

```
# 训练总轮数
epochs = 10
# 模型参数保存目录 (相对当前工作目录)
checkpoint_dir = os.path.join(os.getcwd(), "checkpoints")
# 确保目录存在
os.makedirs(checkpoint_dir, exist_ok=True)

# 记录每个 epoch 的平均 loss
epoch_losses = []
# 记录每个 epoch 的准确率 (若无标签则记录 None)
epoch_accs = []

# 开始训练循环
for epoch in range(epochs):
    # 设置为训练模式
    model.train()
    running_loss = 0.0
    sample_count = 0
    # 逐 batch 训练
    for batch_idx, (images, labels) in enumerate(mydataset_loader):
        # 前向推理得到 logits
        output = model(images)
        # 计算当前 batch 的损失
        loss = loss_fn(output, labels)

        # 清空梯度
        optimizer.zero_grad()
        # 反向传播计算梯度
        loss.backward()
        # 更新参数
        optimizer.step()

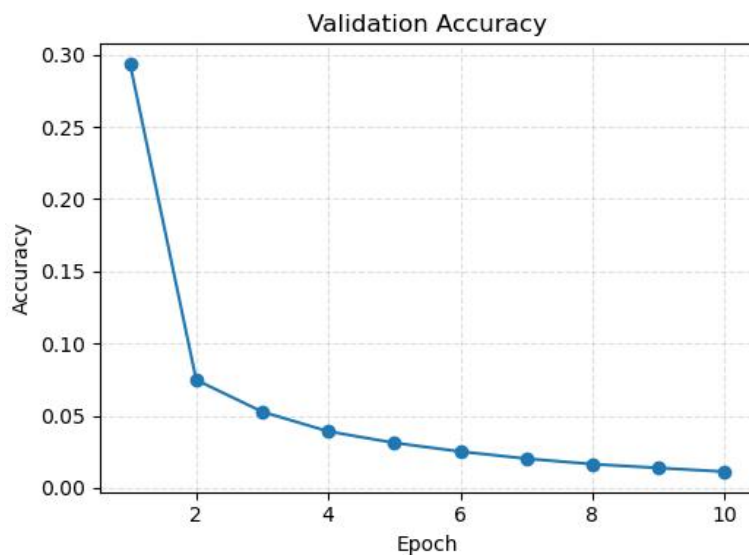
        # 累计 loss (按样本数加权)
        batch_size = labels.size(0)
        running_loss += loss.item() * batch_size
        sample_count += batch_size

    # 输出当前 batch 的 loss
    print(f"Epoch {epoch}, Batch {batch_idx}, Loss: {loss.item():.6f}")

    # 计算并记录该 epoch 的平均 loss
    epoch_loss = running_loss / max(sample_count, 1)
    epoch_losses.append(epoch_loss)

    # 在测试集上做一次评估 (若有标签)
    test_acc = evaluate(model, test_loader)
    if test_acc is not None:
        epoch_accs.append(test_acc)
        print(f"Epoch {epoch}, Test Accuracy: {test_acc * 100:.2f}%")
    else:
        epoch_accs.append(None)

    # 保存该 epoch 的模型参数
    if epoch % 1 == 0:
        ckpt_path = os.path.join(checkpoint_dir, f"model_epoch_{epoch}.pth")
        torch.save(model.state_dict(), ckpt_path)
        print("Saved:", ckpt_path)
```



kaggle_submit_CNN.csv
Complete · now

0.98550

可以得出，最终 CNN 训练结果的准确率 0.98550 高于 FCN 的 0.97196。

1.4 子任务三：实现 Transformer 架构，训练并评估 MNIST_kaggle 数据集的分类效果

Transformer 是一种基于注意力机制的深度学习模型，最初由 Vaswani 等人在 2017 年的论文《Attention is All You Need》中提出。

它彻底改变了自然语言处理(NLP)领域，并逐渐扩展到计算机视觉等几乎所有 AI 方向。

1.4.1 Transformer 介绍

● Transformer 架构

Transformer 的核心思想是完全放弃传统的逐词处理方式（RNN），改用注意力机制让模型一次性看完整个句子，同时判断每个词与其他词的关系，从而实现更快的训练和更强的理解能力。

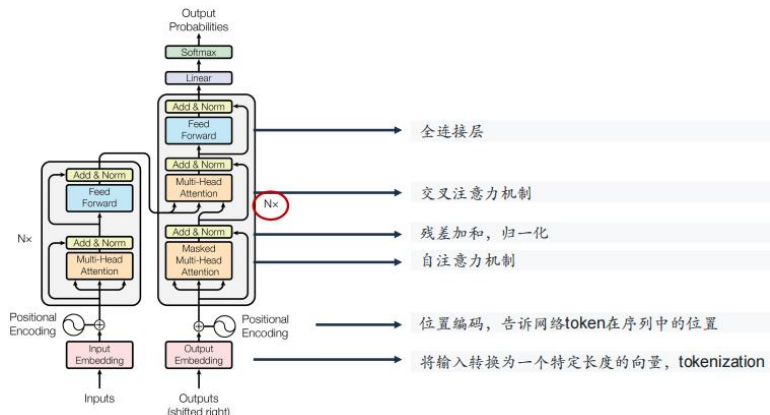
在 Transformer 出现之前，NLP 领域主要依赖 RNN（循环神经网络）系列模型（如 LSTM、GRU），它们按顺序处理文本，存在两个关键缺陷。

RNN 像人类"逐字阅读"一样处理文本，这带来了以下问题：

梯度消失：处理长文本时，模型会"忘记"较早的信息。例如"我昨天在图书馆借了一本关于量子物理的书"中，读到"书"时早已忘记"我"是主语，长距离依赖极难捕捉。

无法并行：RNN 必须按顺序处理每一个词，无法利用 GPU 的并行计算能力，训练超长文本时速度极慢。

Transformer 提出了解决方案:通过自注意力机制，模型同时处理所有词，并动态计算每对词之间的关联强度，彻底解决了上述两个问题。Transformer 由编码器（Encoder）和解码器（Decoder）两大部分组成，各自由多层相同模块堆叠而成。



编码器 (Encoder)

编码器由 N 层相同的模块堆叠而成，每层包含两个子层：

多头自注意力机制 (Multi-Head Self-Attention)：计算输入序列中每个词与其他词的相关性。

前馈神经网络 (Feed-Forward Neural Network)：对每个词进行独立的非线性变换。

每个子层后面都接有残差连接 (Residual Connection) 和层归一化 (Layer Normalization)。

解码器 (Decoder)

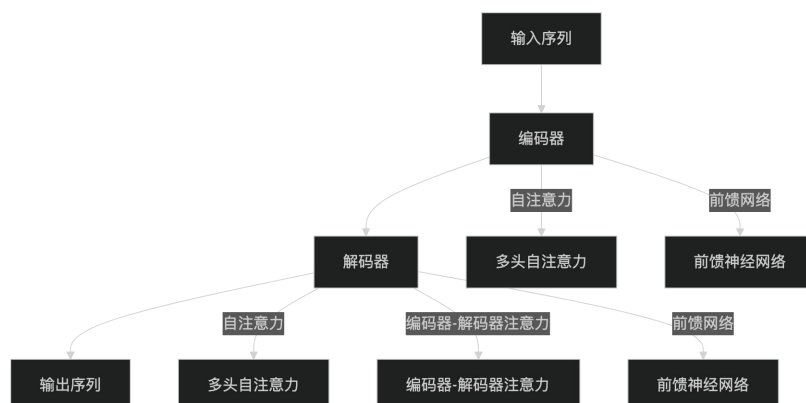
解码器也由 N 层相同的模块堆叠而成，每层包含三个子层：

掩码多头自注意力机制 (Masked Multi-Head Self-Attention)：计算输出序列中每个词与前面词的相关性 (使用掩码防止未来信息泄露)。

编码器-解码器注意力机制 (Encoder-Decoder Attention)：计算输出序列与输入序列的相关性。

前馈神经网络 (Feed-Forward Neural Network)：对每个词进行独立的非线性变换。

同样，每个子层后面都接有残差连接和层归一化。



● 自注意力机制

自注意力机制允许模型同时处理整个输入序列，并动态地为序列中的每个位置分配不同的权重。关于自注意力的细节，详见 <https://zhouyifan.net/2022/09/21/DLS-note-17/>

1.4.2 关键代码及说明

子任务三的流程与子任务一基本相同，主要区别在于

(1) 数据集的构建不同

(2) 神经网络构建不同

接下来主要展示与子任务一不同的地方的代码细节和训练过程及结果

(1) 数据集的构建不同

Transformer 输入为 [1, 28, 28]，后续在模型中做 Patch Embedding。

```
class MyDataset(Dataset):
    # 初始化数据集，支持有标签/无标签两种模式
    def __init__(self, data_dir, labeled=True):
        # 保存数据目录与标注标记
        self.data_dir = data_dir
        self.labeled = labeled
        # samples 保存 (路径, 标签) 或 路径
        self.samples = []
        # 定义统一的图像预处理流程
        self.transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((28, 28)),
            transforms.ToTensor(),
        ])

    # 如果是训练集，读取子目录作为类别
    if self.labeled:
        # 获取所有类别目录名
        class_names = [d for d in os.listdir(data_dir) if os.path.isdir(os.path.join(data_dir, d))]
        # 按数字顺序排序类别目录
        class_names = sorted(class_names, key=lambda x: int(x) if x.isdigit() else x)
        # 构建类别名到标签的映射
        self.class_to_idx = {name: int(name) for name in class_names}
        # 遍历每个类别目录收集样本
        for class_name in class_names:
            class_dir = os.path.join(data_dir, class_name)
            for file_name in os.listdir(class_dir):
                # 只保留图片文件
                if file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                    file_path = os.path.join(class_dir, file_name)
                    label = self.class_to_idx[class_name]
                    self.samples.append((file_path, label))
    else:
        # 测试集无标签，仅保存图片路径
        self.class_to_idx = {}
        for file_name in os.listdir(data_dir):
            file_path = os.path.join(data_dir, file_name)
            # 只保留图片文件
            if os.path.isfile(file_path) and file_name.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
                self.samples.append(file_path)
```

(2) 神经网络构建不同

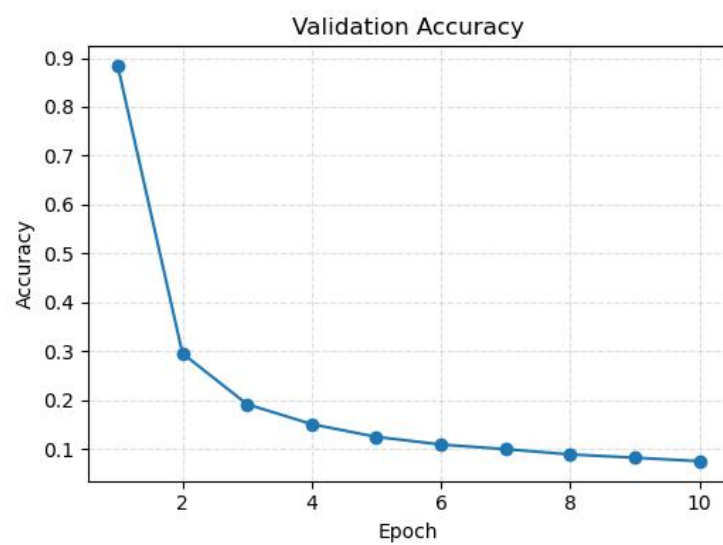
```
class PatchEmbed(nn.Module):
    # 将图像划分为 patch，并映射到 embedding 维度
    def __init__(self, img_size=28, patch_size=4, in_chans=1, embed_dim=64):
        super().__init__()
        if img_size % patch_size != 0:
            raise ValueError("img_size must be divisible by patch_size")
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        # 使用卷积实现 patch 划分与线性映射
        self.proj = nn.Conv2d(in_chans, embed_dim, kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # [B, C, H, W] -> [B, embed_dim, H', W']
        x = self.proj(x)
        # 扁平化并转置，得到 patch 序列 [B, num_patches, embed_dim]
        x = x.flatten(2).transpose(1, 2)
        return x

class MyTransformer(nn.Module):
    # 初始化 Transformer 模型
    def __init__(self, img_size=28, patch_size=4, embed_dim=64, num_heads=4, num_layers=2, mlp_ratio=2):
        super().__init__()
        # Patch Embedding
        self.patch_embed = PatchEmbed(img_size=img_size, patch_size=patch_size, in_chans=1, embed_dim=embed_dim)
        num_patches = self.patch_embed.num_patches
        # 位置编码 (可学习)
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches, embed_dim))
        # Transformer 编码器层
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=embed_dim * mlp_ratio,
            dropout=0.1,
            batch_first=True,
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        # 分类头
        self.classifier = nn.Linear(embed_dim, 10)

    def forward(self, x):
        # 生成 patch token 序列
        x = self.patch_embed(x)
        # 加上位置编码
        x = x + self.pos_embed
        # Transformer 编码
        x = self.encoder(x)
        # 全局平均池化得到图像表示
        x = x.mean(dim=1)
        # 分割输出 logits
        x = self.classifier(x)
        return x
```

训练结果-准确率 0.96196



kaggle_submit_transformer.csv
Complete · now

0.96196

任务二 图神经网络与 UNet 网络

2.1. 任务说明

任务二主要包含两个相互独立、关注不同数据结构的子任务。

子任务一要求使用图神经网络（Graph Neural Networks, GNNs）来处理图结构数据。目标是学习如何构建和训练一个简单的图神经网络模型，以解决图分类或节点分类等任务。

使用的数据集为 Cora 数据集，这是一个常用的图神经网络数据集，共包含 2708 个样本（每个样本代表一篇计算机领域的科学论文），样本之间通过引用与被引用关系构成图的边（Edges）。

每篇论文由一个 1433 维的词向量（特征）表示。词向量中的元素只有 0 和 1（0 表示字典中对应的词不在论文中，1 表示在论文中）。所有的词来源于一个具有 1433 个词的字典，每篇论文都至少引用了一篇其他论文，或者被其他论文引用

所有的论文样本被分为 7 个学术领域类别（包括：基于案例、遗传算法、神经网络、概率方法、强化学习、规则学习、理论）。任务要求通过构建 GCN 网络（利用 GCNConv 算子）对论文节点进行自动分类。



cora.content

每一行由三部分组成，分别是论文的编号，如31336；论文的词向量，一个有1433位的二进制；论文的分类，如Neural_Networks。

cora.cites

5429行，每一行有两个论文编号，第二个编号的论文引用第一个编号的论文。

子任务二区分于传统的图像级分类（多对一映射），本任务要求实现端到端的像素级分类（Pixel-level Classification），即图像的语义分割。

使用数据集是 Carvana 数据集，数据集由成对的图像组成，包括原始车辆照片（如 0cdf5b5d0ce1_01.jpg）和与之严格对应的二值化掩码图（如 .gif 格式的真值 Mask 0cdf5b5d0ce1_01_mask.gif）。



2.2. 子任务一：使用图卷积神经网络（GCN）实现节点分类任务。

2.2.1. GCN 介绍

- GCN 引入

GCN 发表于 2017 年，近两年尤其火爆。要想知道为什么需要 GCN，首先回顾经典的 CNN、RNN 结构：CNN 的核心在于它的 **kernel**，在图片上平移，通过卷积的方式来提取特征。这里关键在于图片结构上的平移不变性：一个小窗口无论移动到图片的哪一个位置，其内部的结构都是一模一样的，因此 CNN 可以实现参数共享。

RNN 的对象是自然语言这样的序列信息，是专门针对序列的结构而设计的，通过各种门的操作，使得序列前后的信息互相影响，从而很好地捕捉序列的特征。

上面讲的图片或者语言，都属于欧式空间的数据，因此才有维度的概念，欧式空间的数据的特点就是结构很规则。但是现实生活中，其实有很多很多不规则的数据结构，典型的就图结构，或称拓扑结构，如社交网络、化学分子结构、知识图谱……

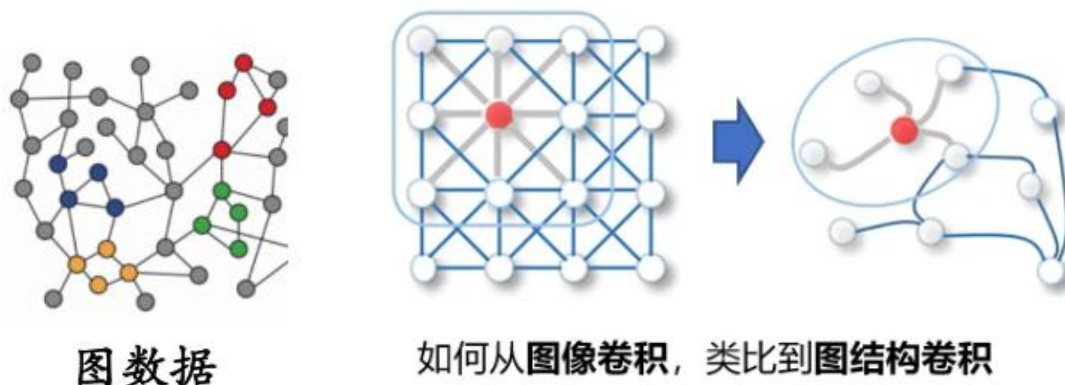
图的结构一般来说是十分不规则的，可以认为是无限维的一种数据，所以它没有平移不变性。每一个节点的周围结构可能都是独一无二的，这种结构的数据，就让传统的 CNN、RNN 瞬间失效。很多学者从上个世纪就开始研究怎么处理这类数据了。这里涌现出了很多方法，例如 GNN、DeepWalk、node2vec 等等……

这里说到的 GCN 跟 CNN 的作用一样，是一个特征提取器，只不过对象是图数据。GCN 的应用广泛，包括：

节点分类（**node classification**），如判断一个用户是否为水军账号

图分类（graph classification），如给定一个化合物的分子结构，判断其是否有害
边预测（link prediction），还可以顺便得到图的嵌入表示（graph embedding），
如在社交网络中进行用户/商品推荐、在生物学领域进行相互作用发现、在知识图谱中
进行实体关系学习

.....



通过理解上一章提到过的图像卷积，来对图结构卷积进行一个类比。

图像卷积的本质其实非常简单，就是将一个像素点周围的像素，按照不同的权重叠加起来，当然这个权重就是我们通常说的卷积核。

其实可以把当前像素点类比做图的节点，而这个节点周围的像素则类比为节点的邻居，从而可以得到图结构卷积的简单的概念：

将一个节点周围的邻居按照不同的权重叠加起来。

而图像上的卷积操作，与图结构上的卷积操作，最大的一个区别就在于：对于图像的像素点来说，它的周围像素点数量其实是固定的；但是对于图而言，节点的邻居数量是不固定的。

● GCN 计算公式

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right)$$

$\hat{A}$ 邻接矩阵

$\hat{D}^{-\frac{1}{2}}$ 与度矩阵相关

$H^{(l)}$ 每一层的节点表示

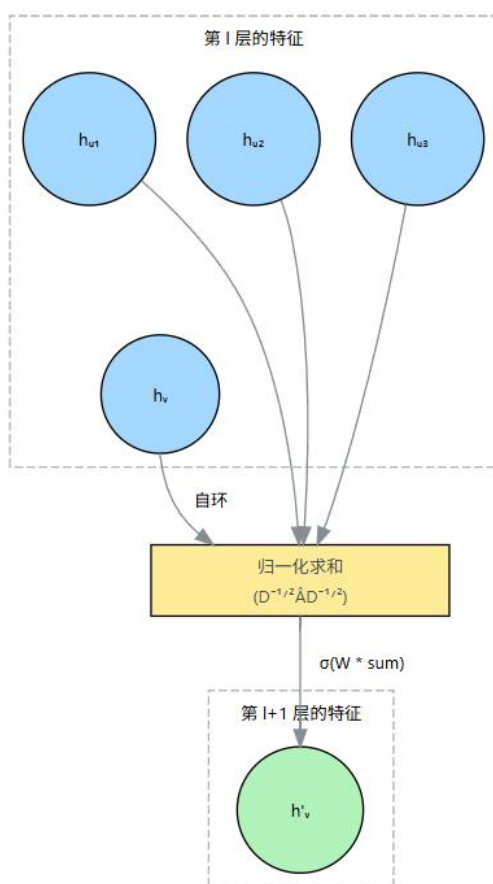
这个方程起初看起来可能很复杂，但每个部分都有其明确且易于理解的作用。我们逐个变量进行分析：

- $H^{(l)}$ 是第 l 层的节点嵌入 (embedding) 矩阵。每一行对应一个节点，每一列代表一个特征。对于第一层 ($l = 0$)，它就是初始节点特征矩阵 X 。
- $W^{(l)}$ 是该层的可训练权重 (weight) 矩阵。这是传统神经网络 (neural network) 中的标准组件。它的任务是对节点特征进行线性变换，使模型能够学习并提取相关的模式。其形状为 [输入特征数, 输出特征数]。
- σ 是非线性激活函数 (activation function) (如 **ReLU**)，应用于每个元素。与其他神经网络一样，它引入了非线性，使模型能够学习更复杂的联系。

GCN 公式中最特别的部分是 $\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}}$ 。这一部分负责核心的图卷积操作，是根据图的结构构建的：

1. $\hat{A} = A + I_N$ ：这是图的邻接矩阵 A 加上单位矩阵 I_N 后的结果，即添加了自环。添加自环是一个简单但操作。它确保了当节点聚合邻居特征时，也会包含自己上一层的特征信息。如果没有这一步，节点自身的表征在更新时会被忽略。
2. $\hat{D}$ ：这是 $\hat{A}$ 的对角度矩阵。每个对角线元素 $\hat{D}_{ii}$ 包含节点 i 的度 (包括其自环)。所有非对角线元素均为零。
3. **对称归一化 (normalization)**：完整项 $\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}}$ 对邻接矩阵进行了对称归一化。这一步对于稳定训练非常。如果只乘以 $\hat{A}$ ，则会简单地累加邻居节点的特征向量 (vector)。然而，这会导致度数非常高或非常低的节点出现问题。高度数节点的嵌入可能会呈指数级增长，而低度数节点的嵌入可能会缩小，从而导致梯度不稳定。通过度矩阵 $\hat{D}^{-\frac{1}{2}}$ 进行归一化，可以有效地对邻居消息取平均，防止节点嵌入的规模受节点度数的影响而产生偏差。

下图说明了单个节点 v 的 GCN 更新过程。它的新特征 h'_v 是通过聚合其自身之前的特征 h_v 以及邻居特征 h_{u1} 、 h_{u2} 和 h_{u3} 计算得出的。



节点 v 的更新涉及对第 l 层局部邻域特征的归一化求和，随后进行线性变换和非线性激活，从而产生其在第 $l+1$ 层的新特征。

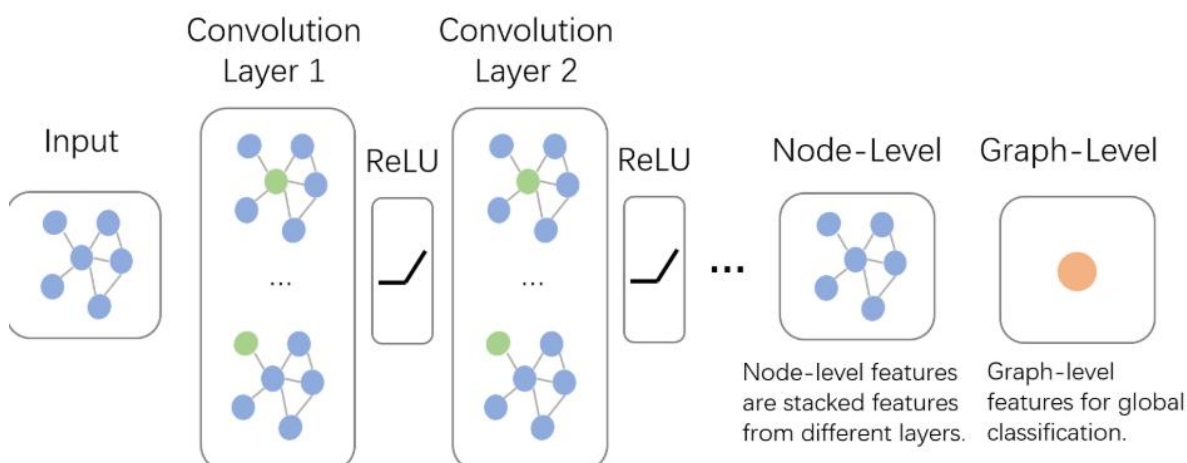
将 GCN 映射到消息传递框架

GCN 公式提供了前一章讨论的 **AGGREGATE**（聚合）和 **UPDATE**（更新）步骤的一个具体实例。GCN 层的优点在于它将这些步骤合并为一个高效的矩阵乘法。

- **聚合**：乘以 $\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}}$ 执行了聚合操作。对于每个节点，此操作收集其邻居（及其自身）的特征，计算归一化 (normalization) 总和，并生成聚合消息。这是一种加权平均，权重 (weight) 由源节点和目标节点的度数决定。
- **更新**：随后乘以权重矩阵 $W^{(l)}$ 并应用激活函数 (activation function) σ 构成了更新步骤。此步骤将聚合的消息转换为节点在下一层的新嵌入 (embedding)。

通用的消息传递方案通常将这些描述为独立的函数，而 GCN 则将它们合并为一次操作。这使其非常高效，尤其是在使用稀疏矩阵乘法库实现时。

● GCN 结构



上面说了图卷积的核心特征，下面我们先来一窥图卷积神经网络的全貌。如下图所示，输入的是整张图，在 **Convolution Layer 1** 里，对每个结点的邻居都进行一次卷积操作，并用卷积的结果更新该结点；然后经过激活函数如 **ReLU**，然后再过一层卷积层 **Convolution Layer 2** 与一层激活函数；反复上述过程，直到层数达到预期深度。

2.2.2. 关键代码及说明

(1) 导入相关依赖

1) 导入相关依赖

```
# 导入操作系统相关的库，用于处理文件路径和系统环境
import os
# 导入 PyTorch 深度学习框架库，用以构建张量和神经网络基础

# 允许 OpenMP 库重复加载，解决绘图时的 Kernel 崩溃问题
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import torch
# 导入 PyTorch 的神经网络层模块（包含如线性层、损失函数等）
import torch.nn as nn
# 导入 PyTorch 神经网络自带函数模块（如激活函数 ReLU 等）
import torch.nn.functional as F
# 导入 NumPy 用于常规的科学计算和 multidimensional array 操作
import numpy as np
```

```
# 导入 Matplotlib 用于训练过程可视化
import matplotlib.pyplot as plt

# 导入 PyG (PyTorch Geometric) 图结构信息存储的 Data 数据类型
from torch_geometric.data import Data
# 导入图结构相关的工具类：用于将有向图转换为无向图
from torch_geometric.utils import to_undirected
# 导入图卷积层（包含标准的GCN模型卷积计算算法）
from torch_geometric.nn import GCNConv

# 打印 PyTorch 版本，确认环境处于可用状态
print("torch version:", torch.__version__)

✓ 0.4s
torch version: 2.8.0+cpu
```

(2) 加载 Cora 数据集

2) 加载Cora数据集

```
# 组合获取本任务所需的节点内容文件和引用边信息文件的本地绝对路径
content_path = os.path.join(os.getcwd(), 'cora', 'cora.content')
cites_path = os.path.join(os.getcwd(), 'cora', 'cora.cites')

# 1. 加载节点特征和标签
print(f"正在解析节点特征和标签{content_path}...")
# 使用 numpy 读取 cora 节点信息文件内全部数据结构信息至 numpy 数组(采用纯文本字符串)
content_data = np.genfromtxt(content_path, dtype=np.dtype(str))
# 赋予每篇论文一个纯数字的模型索引, 建立字符串 ID 到数组下标的快速搜索字典
id_map = {id_str: idx for idx, id_str in enumerate(content_data[:, 0])}
# 将所有图节点最初的字符 ID 单独提取保存
node_ids = content_data[:, 0]

# 忽略第一列 ID 与最后一列分类标签, 将其余的单词矩阵转化为标准的 32 位浮点张量, 作为所
features = torch.tensor(content_data[:, 1:-1].astype(np.float32))

# 单独提取数组最后一列, 保存为标签列
labels_str = content_data[:, -1]
# 为论文分类标签构建映射, 将例如"Neural_Networks"等分类标记转换为不同的整型下标(如: 0)
classes = {label: i for i, label in enumerate(np.unique(labels_str))}
# 根据以上类别映射信息把节点所有分类文字信息更换为 PyTorch 可用于损失计算的包含长整型数据
labels = torch.tensor([classes[label] for label in labels_str], dtype=torch.long)

# 2. 加载边信息 (cora.cites)
print(f"正在解析边信息 {cites_path}...")
# 读取并存放网络中间连接引用的信息(数据按列包含着源论文 ID 和目标引用论文 ID)
cites_data = np.genfromtxt(cites_path, dtype=np.dtype(str))

# 根据之前建立的编号体系(id_map), 将图边界点的绝对论文 ID 转换成系统可操作的连续索引
edges = np.array([[id_map[edge[0]], id_map[edge[1]]] for edge in cites_data])

# 将维度转换为标准的格式: [源节点编号list, 目标节点编号list], 且需保障数据于内存中的连续
edge_index = torch.tensor(edges, dtype=torch.long).t().contiguous()

# 由于节点之间具有无向关系时更利于基础GCN分析, 此处直接把所有的边缘像添加反向边, 形成无
edge_index = to_undirected(edge_index)

# 3. 划分数据集掩码 (Train/Test Mask)
# 计算参与训练、评估的总计图节点数
num_nodes = features.shape[0]
# 获得所有的图节点随机乱序排布号序列
indices = np.random.permutation(num_nodes)
# 从随机序列中选出前 140 号位置设为用于参与正向循环的训练集, 抽取最后
train_idx, test_idx = indices[:140], indices[-1000:]

# 生成长度为总节点数量的掩模张量并在默认初始化时给它们赋予布尔False (
train_mask = torch.zeros(num_nodes, dtype=torch.bool)
# 对应属于训练序列部分节点的下标设定成 True 掩模板
train_mask[train_idx] = True
# 为最终评估保留的类似掩模, 所有默认布尔初值为 False
test_mask = torch.zeros(num_nodes, dtype=torch.bool)
# 根据抽取序列为用作检测用节点开启 True
test_mask[test_idx] = True

# 4. 构建 PyG 的 Data 对象
# 给特征数组(x), 图关系矩阵数组(edge_index)和分类标签数组(y)联
data = Data(x=features, edge_index=edge_index, y=labels)
# 挂载额外属性存放训练以及验证集合节点标记状态, 避免重新传参
data.train_mask = train_mask
data.test_mask = test_mask

# 获得实际有效的词汇数据(如 1433)的宽度
num_node_features = data.num_node_features
# 检测共出现多少实际有效的标签类别总数
num_classes = len(classes)

print(f"\n数据加载完成!")
print(f"节点特征维度: {num_node_features}, 类别数: {num_classes}")
2.3s
```

(3) 构建图神经网络模型 GCNConv

3) 构建图神经网络模型GCNConv

```
class GCN(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        # 继承和调用PyTorch内部神经网络组件的基础配置流程
        super(GCN, self).__init__()
        # 第一层图卷积层: 负责提取基本图的信息聚集。输入即论文本身的 1433 维特征量, 最终转换为我们定义的降维隐藏特征量。
        self.conv1 = GCNConv(in_channels, hidden_channels)
        # 第二层图卷积层: 在隐藏中间态(例如 16 维)特征提取上, 再经过第二遍图内聚类转换出直接对应目标每个分类的分值序列。
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def forward(self, data):
        # 在正向运算时获取传入的所有节点本身数据以及图网络的关系布局数组。
        x, edge_index = data.x, data.edge_index

        # 调用我们先于 __init__ 中搭建的第一个层完成初聚拢特征
        x = self.conv1(x, edge_index)
        # 激活层: 引用极常用的ReLU滤除结果中所有的负成分输出
        x = F.relu(x)
        # 第二层次的卷积运算
        x = self.conv2(x, edge_index)
        # 通过 log 对数分类处理机制把任意权重的向量进行规范收缩成为可判知比率分布结果并供损失运算进行判断(在类别这一维度也就
        return F.log_softmax(x, dim=1)
```

(4) 初始化模型和优化器

4) 初始化模型和优化器

```
# 填入提取到参数信息以真正配置我们用于运行的图神经网络。在此设置其具备隐藏层特征维数为 16。
model = GCN(in_channels=num_node_features, hidden_channels=16, out_channels=num_classes)
# 指定深度学习的神经网络反向误差计算优化器 Adam, 指定默认学习修正步幅(lr: 0.01), 此外使用 weight_decay 权重衰减
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)
# 选择交叉熵熵用来求取各类别实际结果和拟合结论损失比, 从而对神经网络内计算链路节点权重施行奖惩,
criterion = nn.CrossEntropyLoss()
```

(5) 训练与评估模型

5) 训练与评估模型

```
def train():
    # 宣告启动网络训练能力，激活所有层例如 Dropout, BatchNorm 等可能使用到的一些额外变量。
    model.train()
    # 开始每个周期的训练时将过去残留在变量里的所有算术误差清空防止误差迭计算方向。
    optimizer.zero_grad()
    # 提供目前所有顶点初始数据跟图连接配置去完成一次整个网图的前向信息预估推算。
    out = model(data)
    # 利用预先遮盖提取数组 (train_mask) 抽出本次用来练习训练部分的点推算信息然后求其交叉熵值。
    loss = criterion(out[data.train_mask], data.y[data.train_mask])
    # 将模型所得亏损逆向遍历网络以累算出各个层数内对应修正梯度变量
    loss.backward()
    # 发送步骤推行命令要求 Adam 更新层节点变量数值来尝试寻找更小的 Loss 配置
    optimizer.step()
    # 将结果转化至基本 Python 数据内用于对外记录数值展示。
    return loss.item()

def test():
    # 关闭对图权重进行随机微调并使得预测推求趋于明确确切。告知框架本流程意在评估而不是再学了。
    model.eval()
    # 使用训练完后的框架前向跑出结果。
    out = model(data)
    # 对预测分类比里抽取占权重最大的维度标记为其确诊判定的目标分类项类别号。
    pred = out.argmax(dim=1) # 获取每个节点预测出最大可能性的类别
    # 只提取掩膜内的数据来与真实标签进行相等分析并且最终合计数。
    correct = (pred[data.test_mask] == data.y[data.test_mask]).sum().item() # 计算测试集
    # 在正确的全数基础上求总长除式来反馈实际该模式在没有预先透露答案下的总体预言准确度率。
    acc = correct / data.test_mask.sum().item()
    return acc

# 记录训练过程指标
loss_history = []
acc_history = []
acc_epochs = []

# 定义该 AI 要用所有的样本执行一共 200 次轮询遍历计算练习。
for epoch in range(200):
    # 进行内部训练周期一次并且抽取返回这期间内的计算不确定丢失亏损。
    loss = train()
    loss_history.append(loss)
    # 由于频率 Epoch 输出会有很高占幅，每走过整除这 20 倍数轮番进行一次评判测试以展示进步量。
    if (epoch + 1) % 20 == 0:
        # 获取当下状态中模型测算出正确的百分比。
        acc = test()
        acc_history.append(acc)
        acc_epochs.append(epoch + 1)
        # 清晰化呈现控制台输出用于掌握训练的进度。
        print(f'Epoch: {epoch + 1:03d}, Loss: {loss:.4f}, Test Accuracy: {acc:.4f}')

# 训练过程与结果可视化
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(range(1, len(loss_history) + 1), loss_history, color="tab:blue")
```

✓ 2.0s

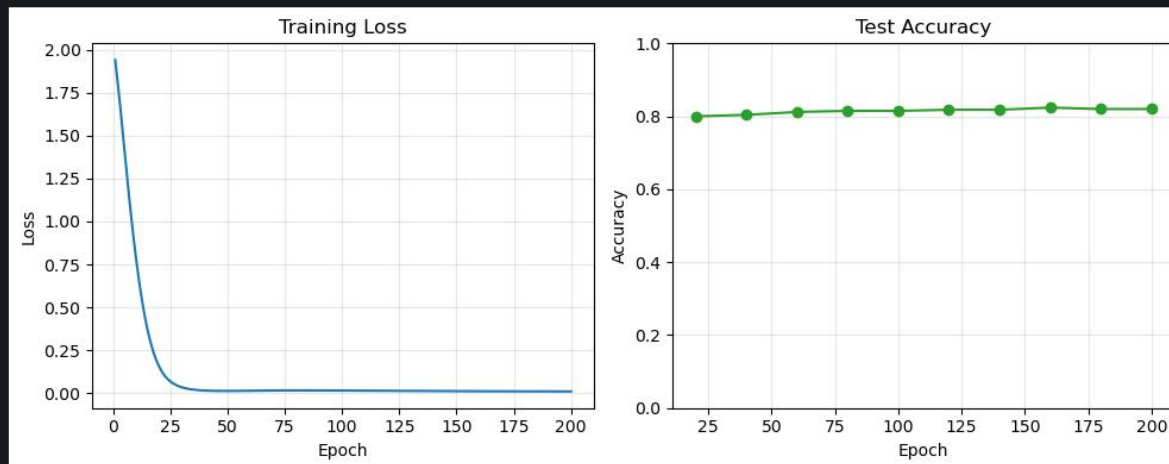
```
# 训练过程与结果可视化
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(range(1, len(loss_history) + 1), loss_history, color="tab:blue")
axes[0].set_title("Training Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].grid(True, alpha=0.3)

axes[1].plot(acc_epochs, acc_history, marker="o", color="tab:green")
axes[1].set_title("Test Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_ylim(0, 1)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

训练结果

```
Epoch: 020, Loss: 0.1595, Test Accuracy: 0.8000
Epoch: 040, Loss: 0.0149, Test Accuracy: 0.8040
Epoch: 060, Loss: 0.0135, Test Accuracy: 0.8120
Epoch: 080, Loss: 0.0158, Test Accuracy: 0.8150
Epoch: 100, Loss: 0.0142, Test Accuracy: 0.8150
Epoch: 120, Loss: 0.0125, Test Accuracy: 0.8180
Epoch: 140, Loss: 0.0115, Test Accuracy: 0.8180
Epoch: 160, Loss: 0.0107, Test Accuracy: 0.8240
Epoch: 180, Loss: 0.0100, Test Accuracy: 0.8200
Epoch: 200, Loss: 0.0095, Test Accuracy: 0.8200
```



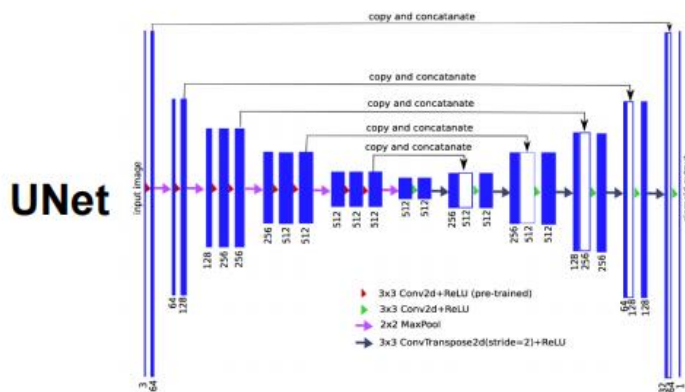
2.3. 子任务二：使用 U-Net 模型实现图像分割任务

2.3.1. U-Net 介绍

UNet 是深度学习在计算机视觉(CV)领域最经典的图像分割(Image Segmentation)网络架构之一，由 Olaf Ronneberger 等人于 2015 年提出，最初专门用于医疗图像分割（如细胞轮廓检测、器官分割）。

传统的卷积神经网络（如 VGG、ResNet）通常会不断下采样（Downsampling）以提取高维的抽象特征，最终通过全连接层输出一个图像级的分类标签。但语义分割要求输出与原图分辨率完全一致的像素级掩码（Mask），这意味着网络不仅要懂“图里有什么”（语义信息），还要懂“东西在哪里”（精确的空间位置信息）。

为了同时兼顾这两点，UNet 创造性地设计成了完全对称的 U 型结构：



左半部分：收缩路径（Contracting Path / Encoder 编码器）

作用：捕获图像的上下文语义信息（Context）。

结构：由重复的卷积层（通常伴随 ReLU 激活）和最大池化层（MaxPool2d）组成。每经过一次池化，图像的宽高减半（下采样），但通道数翻倍，从而深入提取高级别的抽象特征。

右半部分：扩张路径（Expanding Path / Decoder 解码器）

作用：精确定位（Localization），将低分辨率的特征图恢复到原图尺寸。

结构：由重复的转置卷积（ConvTranspose2d）或上采样层组成。每经过一次上采样，图像的宽高翻倍，通道数减半，逐步构建出密集的像素级预测。

2.3.2. 关键代码及说明

导入库与数据集过程省略，在此只展示网络构建和训练过程。

(1) U-Net 构建

```
>class DoubleConv(nn.Module): ...

>class Down(nn.Module): ...

>class Up(nn.Module): ...

class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=1, base_channels=64):
        super().__init__()
        self.inc = DoubleConv(in_channels, base_channels)
        self.down1 = Down(base_channels, base_channels * 2)
        self.down2 = Down(base_channels * 2, base_channels * 4)
        self.down3 = Down(base_channels * 4, base_channels * 8)
        self.down4 = Down(base_channels * 8, base_channels * 16)
        self.up1 = Up(base_channels * 16, base_channels * 8)
        self.up2 = Up(base_channels * 8, base_channels * 4)
        self.up3 = Up(base_channels * 4, base_channels * 2)
        self.up4 = Up(base_channels * 2, base_channels)
        self.outc = nn.Conv2d(base_channels, out_channels, kernel_size=1)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        return self.outc(x)
```

(2) 训练过程

```
model = UNet(in_channels=3, out_channels=1, base_channels=64).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

epochs = 10
history = {"train_loss": [], "val_loss": [], "val_dice": [], "val_iou": []}

for epoch in range(1, epochs + 1):
    train_loss = train_one_epoch(model, train_loader, optimizer, device)
    val_loss, val_dice, val_iou = evaluate(model, val_loader, device)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_dice"].append(val_dice)
    history["val_iou"].append(val_iou)
    print(
        f"Epoch {epoch:02d} | "
        f"Train Loss: {train_loss:.4f} | "
        f"Val Loss: {val_loss:.4f} | "
        f"Dice: {val_dice:.4f} | "
        f"IoU: {val_iou:.4f}"
    )

Epoch 01 | Train Loss: 0.1494 | Val Loss: 0.1349 | Dice: 0.9439 | IoU: 0.9084
```

U-Net 的训练非常耗时，所以只训练了前几轮

(3) 训练过程可视化

7) 结果可视化（预测示例）

生成 代码 Markdown

```
model.eval()
images, masks = next(iter(val_loader))
images = images.to(device)
masks = masks.to(device)

with torch.no_grad():
    logits = model(images)
    probs = torch.sigmoid(logits)
    preds = (probs > 0.5).float()

n = min(3, images.size(0))
fig, axes = plt.subplots(n, 3, figsize=(9, 3 * n))
if n == 1:
    axes = [axes]

for i in range(n):
    axes[i][0].imshow(images[i].cpu().permute(1, 2, 0))
    axes[i][0].set_title("Image")
    axes[i][0].axis("off")

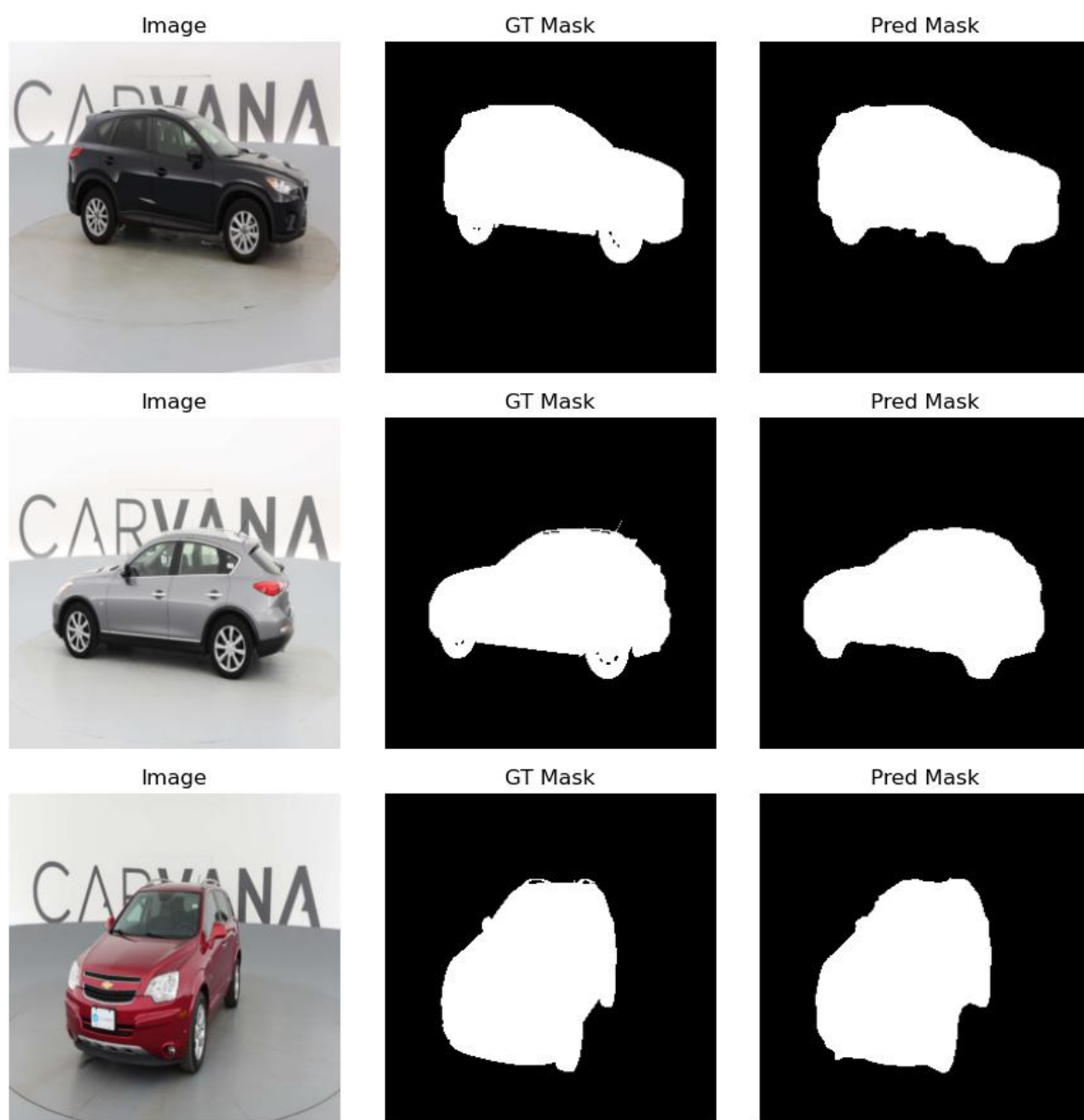
    axes[i][1].imshow(masks[i].cpu().squeeze(0), cmap="gray")
    axes[i][1].set_title("GT Mask")
    axes[i][1].axis("off")

    axes[i][2].imshow(preds[i].cpu().squeeze(0), cmap="gray")
    axes[i][2].set_title("Pred Mask")
    axes[i][2].axis("off")

plt.tight_layout()
plt.show()

✓ 2.7s
```

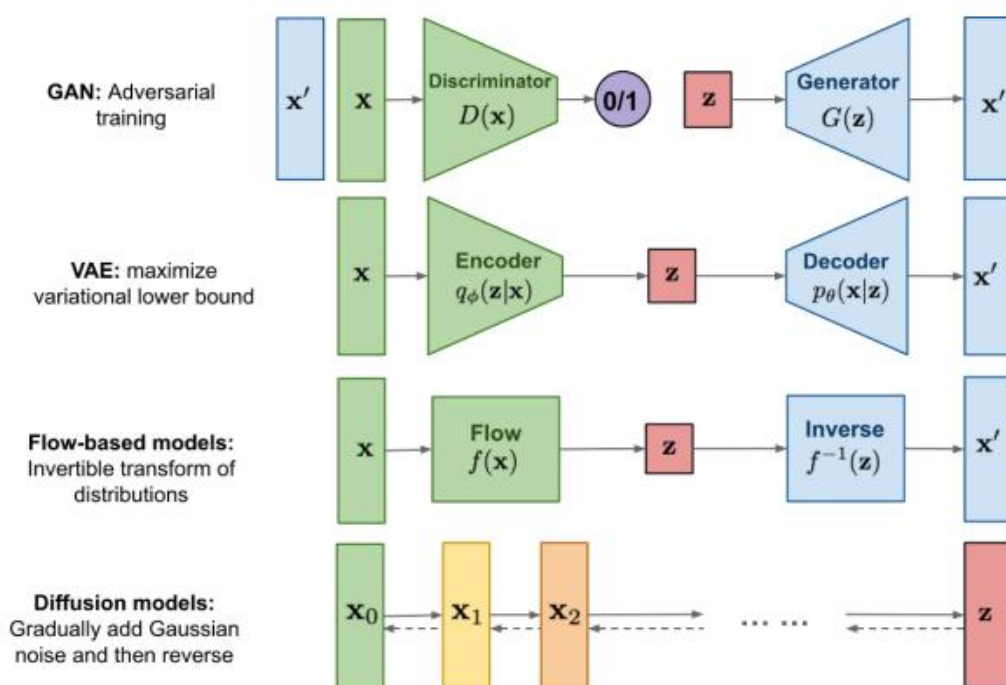
输出结果为



任务三 生成模型

4.1. 任务说明

本任务旨在通过端到端的代码编写与模型训练，目标是使得模型能学会从已知真实数据分布中学习并生成全新、逼真未见样本的非线性映射机理。任务重点在于对比不同生成模型（VAE、GAN、Diffusion Models）在图像生成质量、训练稳定性和特征流控上的异同。



4.2. 子任务一 构建 VAE 网络生成手写数字

4.2.1. VAE 介绍

VAE（Variational Autoencoder，变分自编码器）是深度学习中非常经典且应用广泛的生成模型（Generative Model）之一，由 Diederik Kingma 和 Max Welling 于 2013 年提出。

- 自编码器（Autoencoder, AE）

自编码器（Autoencoder, AE）是一种无监督学习的神经网络，通过学习将输入数据压缩到低维潜在空间，再从压缩表示重构原始数据。

自编码器广泛应用于数据降维、特征提取、异常检测、图像去噪、生成模型等场

景。

自编码器的基本结构包含三个部分：

编码器（Encoder）：将输入数据 x 映射到低维潜在表示 z

潜在空间（Latent Space）：编码器输出的低维向量，也称为瓶颈层

解码器（Decoder）：将潜在表示 z 重构为输出 $\hat{x}$

自编码器的目标是让输出 $\hat{x}$ 尽可能接近输入 x ：

$$\min_{\theta, \phi} \frac{1}{n} \sum_{i=1}^n \|x_i - D_{\phi}(E_{\theta}(x_i))\|^2$$

其中 θ 是编码器参数， ϕ 是解码器参数。

自编码器通过强制数据通过比输入维度更小的瓶颈层，从而学习数据的压缩表示。这种压缩保留了数据的主要信息。实现了降维效果。

- 变分自编码器（Variational Autoencoder, VAE）

而变分自编码器（Variational Autoencoder, VAE）是一种生成模型，它将数据编码为潜在空间中的概率分布，而非固定向量。这使得我们可以从潜在空间中采样生成新数据。

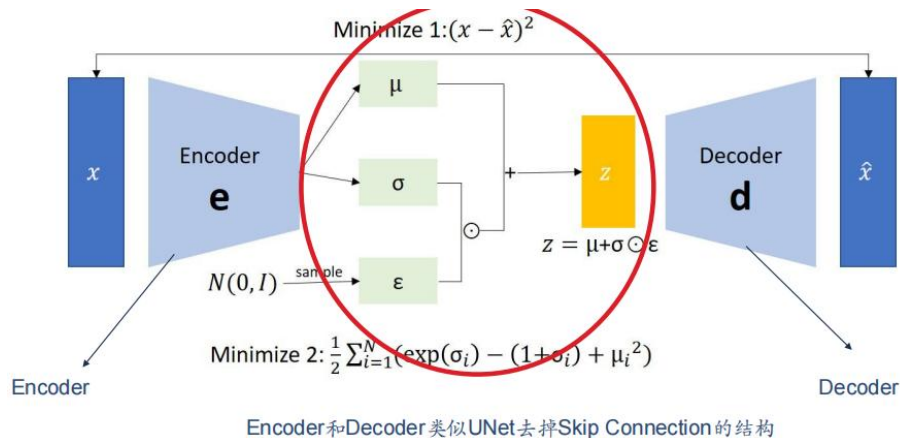
VAE 的关键创新是学习潜在变量的概率分布：

编码器输出均值 μ 和标准差 σ ；从正态分布 $N(\mu, \sigma)$ 中采样得到潜在向量 z ；解码器从 z 重构数据；

为了实现可微的采样过程，使用了重参数化技巧（Reparameterization Trick）：

$$z = \mu + \sigma \cdot \epsilon \quad \epsilon \sim N(0,1)$$

VAE 的潜在空间是连续的，可以在潜在空间中进行插值，生成平滑过渡的图像。但 VAE 生成的图像通常较模糊，这是因为它优化的是下界而非精确的对数似然。



4.2.2. 关键代码及说明

(1) 导入必要的库

1. 导入必要的库

```
import os
# 允许 OpenMP 库重复加载, 解决绘图时的 Kernel 崩溃问题
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# 检查是否有 GPU 可用
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"当前使用的设备: {device}")

# 超参数设置
batch_size = 128
epochs = 15
learning_rate = 1e-3

当前使用的设备: cpu
```

(2) 加载数据集

2. 加载MNIST数据集

```
# 数据预处理: 转换为 Tensor
transform = transforms.Compose([transforms.ToTensor()])

# 加载 MNIST 训练集
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size, shuffle=True)
```

[2]

(3) 构建 VAE 模型 定义 encoder 和 decoder, 给出 z 的得到方式。

3. 构建VAE模型

```
class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super(VAE, self).__init__()

        # Encoder (编码器)
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        # 映射到均值和对数方差
        self.fc2_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc2_logvar = nn.Linear(hidden_dim, latent_dim)

        # Decoder (解码器)
        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        h1 = F.relu(self.fc1(x))
        return self.fc2_mu(h1), self.fc2_logvar(h1)

    # 课件 P35: 重参数化技巧
    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar) # 计算标准差
        eps = torch.randn_like(std) # 从标准正态分布中采样噪声
        return mu + eps * std # 结合均值和方差返回 z

    def decode(self, z):
        h3 = F.relu(self.fc3(z))
        # 使用 sigmoid 确保输出在 0~1 之间, 对应图像像素并方便计算 BCE loss
        return torch.sigmoid(self.fc4(h3))

    def forward(self, x):
        # 展平图像
        x = x.view(-1, 784)
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

# 实例化模型并移动到 device
model = VAE().to(device)
```

(4) 定义损失函数和优化器

4. 定义损失函数和优化器

```
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# 课件 P35: VAE 损失函数 = 重构损失 (BCE) + KL 散度约束 (KLD)
def loss_function(recon_x, x, mu, logvar):
    # 使用二值交叉熵作为重构损失 (sum 所有像素点)
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 784), reduction='sum')

    # KL散度: 使隐变量分布接近标准正态分布
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return BCE + KLD
```

(5) 训练模型

5. 训练模型

```
model.train()
loss_history = []

print("开始训练 VAE...")
for epoch in range(1, epochs + 1):
    train_loss = 0
    for batch_idx, (data, _) in enumerate(train_loader):
        data = data.to(device)

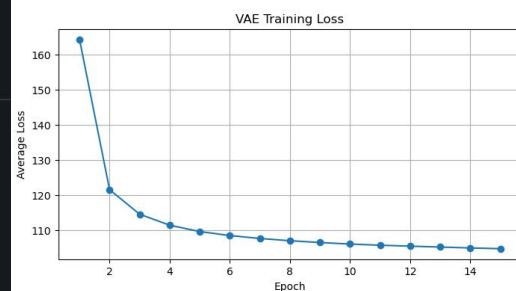
        optimizer.zero_grad()
        # 前向传播
        recon_batch, mu, logvar = model(data)
        # 计算损失
        loss = loss_function(recon_batch, data, mu, logvar)
        # 反向传播
        loss.backward()
        train_loss += loss.item()
        optimizer.step()

    avg_loss = train_loss / len(train_loader.dataset)
    loss_history.append(avg_loss)
    print(f'====> Epoch: {epoch} Average loss: {avg_loss:.4f}')

# 绘制 Loss 曲线
plt.figure(figsize=(8, 4))
plt.plot(range(1, epochs + 1), loss_history, marker='o')
plt.title('VAE Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Average Loss')
plt.grid()
plt.show()
```

开始训练 VAE...

```
====> Epoch: 1 Average loss: 164.2426
====> Epoch: 2 Average loss: 121.4524
====> Epoch: 3 Average loss: 114.4614
====> Epoch: 4 Average loss: 111.3404
====> Epoch: 5 Average loss: 109.5306
====> Epoch: 6 Average loss: 108.3933
```



(6) 生成新样本并可视化

6. 生成新样本并可视化

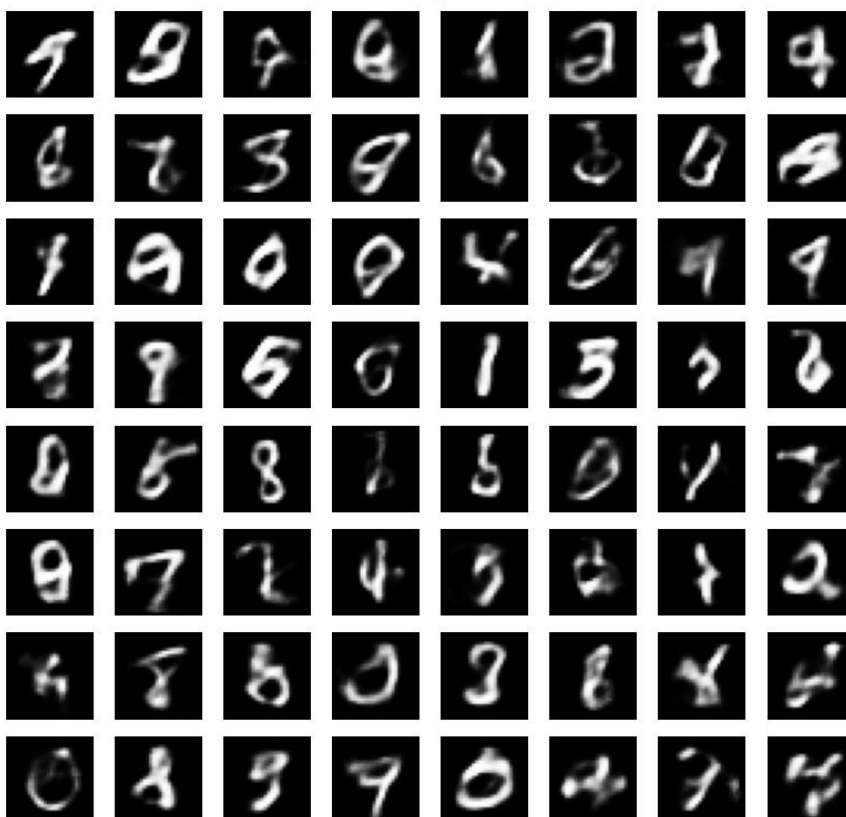
```
model.eval()
with torch.no_grad():
    # 随机采样 64 个隐变量 z (从标准正态分布 N(0,1) 中)
    z = torch.randn(64, 20).to(device)

    # 通过 Decoder 生成图像
    sample = model.decode(z).cpu()

    # 可视化生成的图像
    fig, axes = plt.subplots(8, 8, figsize=(8, 8))
    for i, ax in enumerate(axes.flatten()):
        ax.imshow(sample[i].view(28, 28), cmap='gray')
        ax.axis('off')

plt.suptitle("Generated Digits from VAE", fontsize=16)
plt.tight_layout()
plt.show()
```

Generated Digits from VAE

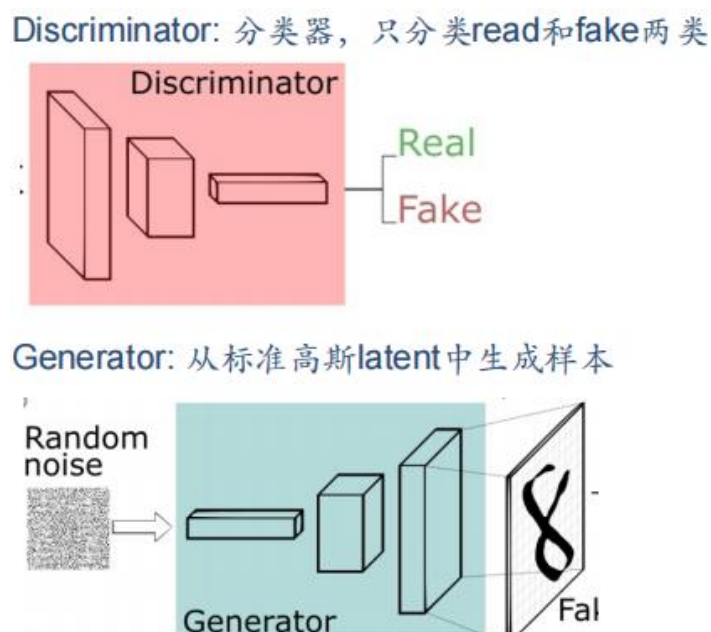


VAE 的训练比较容易，但是可以看出，生成的图片比较模糊。这是由于损失函数采用了均值回归的概率假设，VAE 倾向于对像素取平均，导致生成的图像细节（如边缘、纹理）不如 GAN 和扩散模型（Diffusion Models）那样锐利。

4.3. 子任务二 构建 GAN 网络生成手写数字

4.3.1. GAN 介绍

生成对抗网络（Generative Adversarial Network, GAN）是深度学习中最具创意的模型架构之一。它通过让两个神经网络相互对抗、相互学习，最终能够生成非常逼真的数据。GAN 广泛应用于图像生成、风格迁移、数据增强等场景。



GAN 的核心思想来源于博弈论中的"零和博弈"。它包含两个相互对抗的网络：

生成器（Generator）：学习生成假数据，目标是让判别器无法区分生成数据与真实数据；

判别器（Discriminator）：学习区分真实数据与生成数据，目标是尽可能准确判断照片的真假；

两者在训练过程中相互对抗、不断提升，最终达到纳什均衡状态。

GAN 的训练目标可以表示为以下 minimax 游戏：

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

其中

G 表示生成器网络

D 表示判别器网络

x 表示真实数据

z 表示随机噪声向量（通常服从标准正态分布）

$G(z)$ 表示生成器根据噪声生成的假数据

GAN 的训练分为两个阶段:

第一阶段: 训练判别器

固定生成器, 提升判别器的分辨能力:

$$\max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

第二阶段: 训练生成器

固定判别器, 提升生成器的欺骗能力:

$$\min_G \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

GAN 极不稳定, 极易发生梯度消失或模式崩塌, 本次在实际训练中也花费了很多精力调整以让生成器和判别器“势均力敌”。

4.3.2. 关键代码及说明

导入库和加载 MNIST 数据集和子任务一类似, 故略去, 重点展示网络构建和训练过程。

(1) GAN 网络构建

```
# 生成器: 从噪声生成图像
class Generator(nn.Module):
    def __init__(self, noise_dim, img_dim=784):
        super(Generator, self).__init__()
        self.gen = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.BatchNorm1d(256), # ----- 新增 BN 层
            nn.LeakyReLU(0.2),

            nn.Linear(256, 512),
            nn.BatchNorm1d(512), # ----- 新增 BN 层
            nn.LeakyReLU(0.2),

            nn.Linear(512, 1024),
            nn.BatchNorm1d(1024), # ----- 新增 BN 层
            nn.LeakyReLU(0.2),

            nn.Linear(1024, img_dim),
            nn.Tanh() # 将输出压缩到 [-1, 1] 区间, 匹配归一化后的图像像素
        )

    def forward(self, x):
        return self.gen(x)

# 判别器: 区分真实图像和虚假图像
class Discriminator(nn.Module):
    def __init__(self, img_dim=784):
        super(Discriminator, self).__init__()
        self.disc = nn.Sequential(
            nn.Linear(img_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(128, 1),
            nn.Sigmoid() # 输出 0~1 的概率, 代表是真图的概率
        )

    def forward(self, x):
        return self.disc(x)
```

值得说明的是, 本次任务中 G 和 D 经过了多次修改和重构, 主要是因为训练过程输出的 loss 表现出来的问题, 经历的过程包括判别器 (Discriminator) 过于强大, 导致生成器 (Generator) 梯度消失。经过调参之后仍然表现为 D 过强, 于是对 D 的结构进行了剪枝, 但在这之后 G 过强, 于是又修改了 G 的结构, 反复修改之后, 最终 loss 输出是我们想要的结果了: 范围稳定且相互对抗, 但是发现生成的图片很糟糕, 是灰色的模糊图像, 这可能是实质上是生成器发现了捷径: 只要输出一种特定的灰色模糊

图像，已经被我们之前“过度削弱”的判别器就会认不出来，只能每次都猜 0.5。

最后给 G 加入 BN 层，并且修改数据范围在-1 到 1 之间，使用 tanh 激活函数，这样使得梯度传递更加稳定，再将 D 又还原成削弱前的样子，最终模型稳定。

(2) 训练过程

```
print("开始训练 GAN...")
for epoch in range(epochs):
    for i, (imgs, _) in enumerate(dataloader):
        batch_size_cur = imgs.size(0)

        # 将图像展平为向量，并移动到 GPU/CPU
        real_imgs = imgs.view(batch_size_cur, -1).to(device)

        # 创建真实标签 (1) 和虚假标签 (0)
        real_labels = (torch.ones(batch_size_cur, 1)).to(device) # 使用 0.9 而不是 1
        fake_labels = (torch.zeros(batch_size_cur, 1)).to(device)

        # =====
        # 1. 训练判别器 (Discriminator)
        # =====
        optimizer_D.zero_grad()

        # 1.1 真实样本的损失
        outputs_real = D(real_imgs)
        d_loss_real = criterion(outputs_real, real_labels)
        d_loss_real.backward()

        # 1.2 虚假样本的损失
        noise = torch.randn(batch_size_cur, noise_dim).to(device)
        fake_imgs = G(noise)
        # 使用 .detach() 是为了避免在更新 D 时计算 G 的梯度
        outputs_fake = D(fake_imgs.detach())
        d_loss_fake = criterion(outputs_fake, fake_labels)
        d_loss_fake.backward()

        d_loss = d_loss_real + d_loss_fake
        optimizer_D.step()
```

```
# =====
# 2. 训练生成器 (Generator)
# =====
optimizer_G.zero_grad()

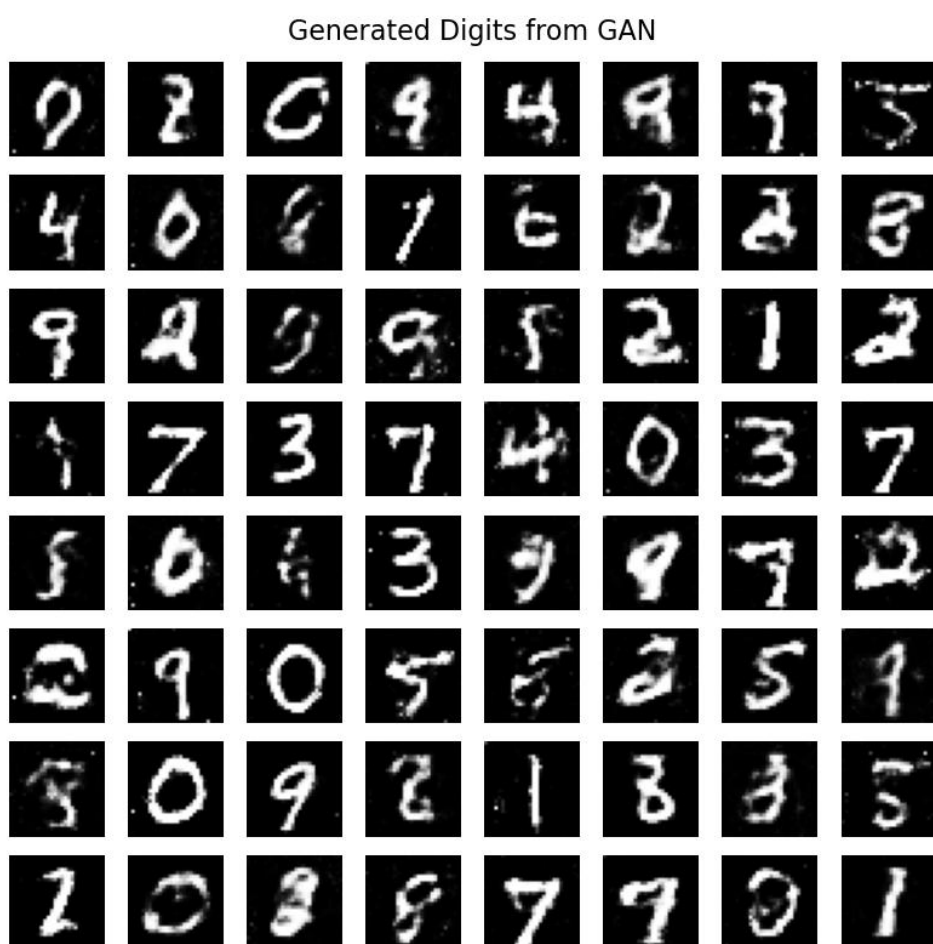
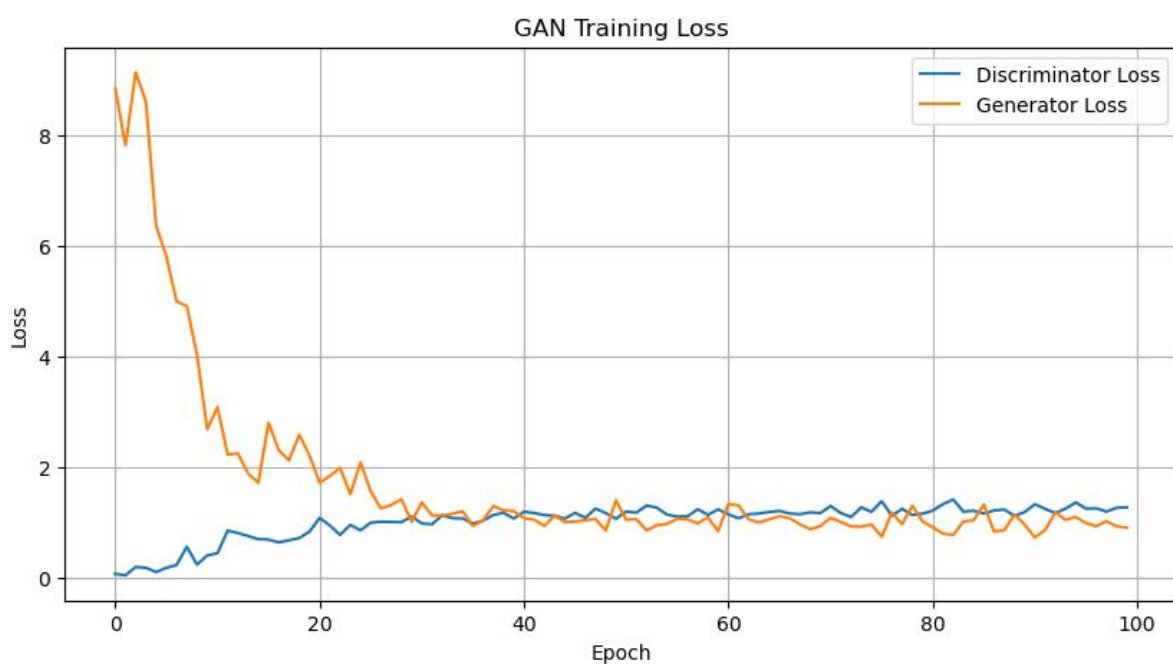
# 生成器希望判别器将自己生成的虚假样本判定为真实 (标签给 1)
outputs = D(fake_imgs)
g_loss = criterion(outputs, real_labels)
g_loss.backward()

optimizer_G.step()

# 记录每个 epoch 最后的 loss
d_losses.append(d_loss.item())
g_losses.append(g_loss.item())

if (epoch + 1) % 5 == 0 or epoch == 0:
    print(f"Epoch [{epoch+1}/{epochs}] | D Loss: {d_loss.item():.4f} | G Loss: {g_loss.item():.4f}")

# 绘制 Loss 曲线
plt.figure(figsize=(10, 5))
plt.plot(d_losses, label='Discriminator Loss')
plt.plot(g_losses, label='Generator Loss')
plt.title('GAN Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.show()
```



可以看出，GAN 生成的图像质量高于 VAE，但是训练过程很不稳定，非常难以训练。

4.4. 子任务三 构建扩散模型生成手写数字

4.4.1. 扩散模型介绍

扩散模型（Diffusion Models）是近年来深度学习领域最火热、最前沿的生成模型，也是诸如 Midjourney、Stable Diffusion 和 Dall-E 3 等顶尖 AI 绘图工具的核心底层技术。

扩散模型在数学上由两个互逆的阶段组成：

① 前向过程（Forward Process，又称加噪过程）

过程：是一个确定性的马尔可夫链。系统在每一个时间步 t （如 $t = 1, 2, \dots, T$ ），都在前一步的图像中注入一点点微小的标准高斯噪声。

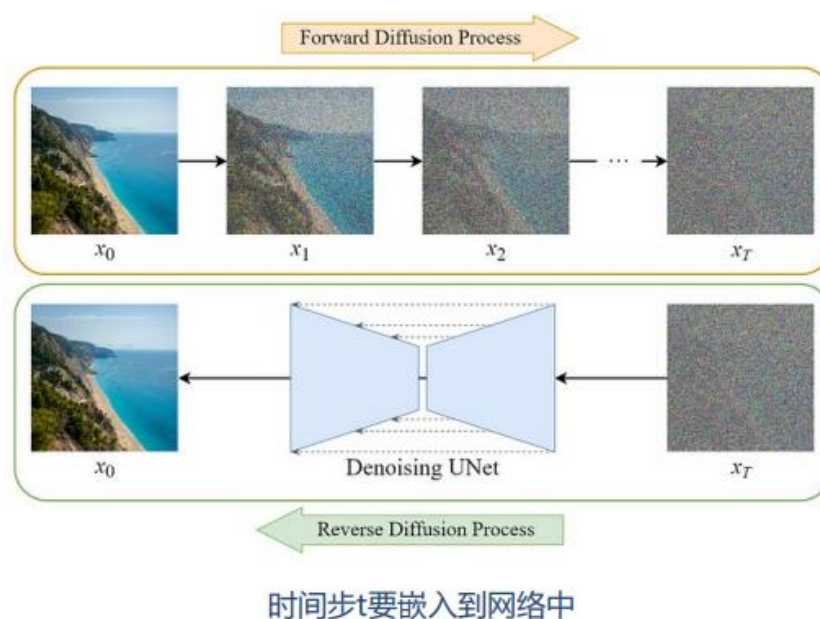
结果：当 T 足够大时（例如多步迭代），图像中原本含有的结构信息（如手写数字的笔画）会被彻底淹没，最终变成一个纯高斯噪声流。

由于高斯分布的优良性质，前向加噪不需要真的逐步计算。可以通过闭式方程，直接计算出任意时间步 t 加噪后的图像。

② 反向过程（Reverse Process，又称去噪/生成过程）

过程：这是由神经网络（如 MiniUNet）控制的猜测过程。我们从一个纯高斯噪声开始，让网络去预测当前时间步图像里包含的“噪声成分”是多少，然后用当前图像减去预测的噪声，逐步剔除。

结果：从无序的混沌（纯噪声）中，一步一步自适应还原出极度锐利、逼真的手写数字或复杂图像。



4.4.2. 关键代码及说明

导入库和加载 MNIST 数据集和子任务一类似，故略去，重点展示网络构建和训练过程。

(1) 前向扩散与反向去噪过程

3. 前向扩散过程 (q)

按课件设置线性噪声调度，定义 $q(x_t|x_0)$ 。

```
def cosine_beta_schedule(timesteps, s=0.008):
    steps = timesteps + 1
    x = torch.linspace(0, timesteps, steps, device=device)
    alphas_bar = torch.cos(((x / timesteps) + s) / (1 + s) * torch.pi / 2) ** 2
    alphas_bar = alphas_bar / alphas_bar[0]
    betas = 1 - (alphas_bar[1:] / alphas_bar[:-1])
    return torch.clamp(betas, 1e-4, 0.999)

betas = cosine_beta_schedule(timesteps)
alphas = 1.0 - betas
alphas_bar = torch.cumprod(alphas, dim=0)

def extract(a, t, x_shape):
    batch_size = t.size(0)
    out = a.gather(0, t).float()
    return out.view(batch_size, *((1,) * (len(x_shape) - 1)))

def q_sample(x0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)
    sqrt_ab = torch.sqrt(extract(alphas_bar, t, x0.shape))
    sqrt_one_minus_ab = torch.sqrt(1.0 - extract(alphas_bar, t, x0.shape))
    return sqrt_ab * x0 + sqrt_one_minus_ab * noise
```

✓ 0.0s

4. 构建去噪模型

输入 x_t 和时间步 t , 预测噪声 $\epsilon_\theta(x_t, t)$.

```
class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
        self.time_proj = nn.Linear(time_dim, out_channels)

    def forward(self, x, t_emb):
        h = F.relu(self.conv1(x))
        h = h + self.time_proj(t_emb).view(t_emb.size(0), -1, 1, 1)
        h = F.relu(self.conv2(h))
        return h
```

```
class UNetSmall(nn.Module):
    def __init__(self, time_dim=128):
        super().__init__()
        self.time_emb = nn.Sequential(
            nn.Embedding(timesteps, time_dim),
            nn.Linear(time_dim, time_dim),
            nn.ReLU(inplace=True),
            nn.Linear(time_dim, time_dim),
        )
        self.down1 = ConvBlock(1, 64, time_dim)
        self.down2 = ConvBlock(64, 128, time_dim)
        self.down3 = ConvBlock(128, 256, time_dim)
        self.pool = nn.MaxPool2d(2)
        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.conv_up2 = ConvBlock(256, 128, time_dim)
        self.conv_up1 = ConvBlock(128, 64, time_dim)
        self.out = nn.Conv2d(64, 1, kernel_size=1)

    def forward(self, x, t):
        t_emb = self.time_emb(t)
        x1 = self.down1(x, t_emb)
        x2 = self.down2(self.pool(x1), t_emb)
        x3 = self.down3(self.pool(x2), t_emb)
        x = self.up2(x3)
        x = torch.cat([x2, x], dim=1)
        x = self.conv_up2(x, t_emb)
        x = self.up1(x)
        x = torch.cat([x1, x], dim=1)
        x = self.conv_up1(x, t_emb)
        return self.out(x)

model = UNetSmall().to(device)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
✓ 0.0s
```

(2) 训练过程

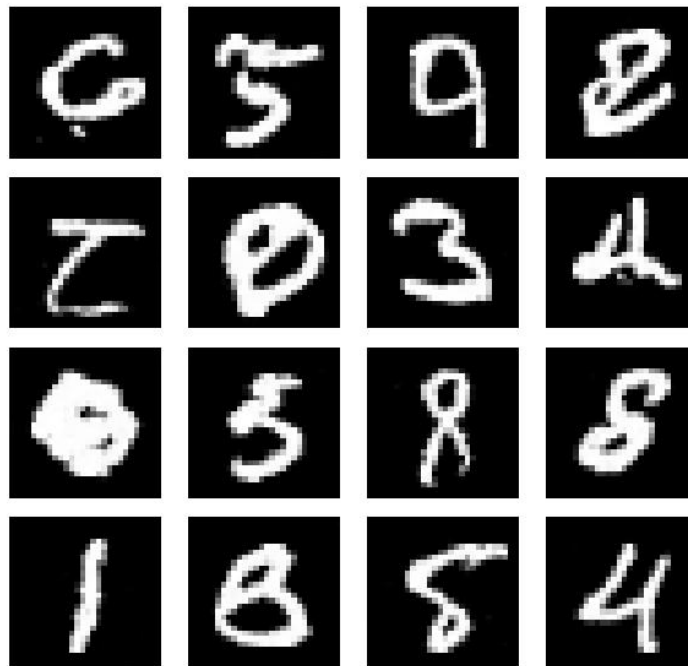
5. 训练模型

目标是让模型预测前向扩散中加入的噪声。

```
loss_history = []
print("开始训练扩散模型...")
for epoch in range(1, epochs + 1):
    model.train()
    total_loss = 0.0
    for x0, _ in train_loader:
        x0 = x0.to(device)
        t = torch.randint(0, timesteps, (x0.size(0),), device=device).long()
        noise = torch.randn_like(x0)
        x_t = q_sample(x0, t, noise)
        pred_noise = model(x_t, t)
        loss = F.mse_loss(pred_noise, noise)
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        total_loss += loss.item() * x0.size(0)
    avg_loss = total_loss / len(train_loader.dataset)
    loss_history.append(avg_loss)
    print(f"Epoch {epoch:02d} | Loss: {avg_loss:.6f}")
⊗ 437m 52.8s
```

可以看出，扩散模型的训练非常耗时，由于设备和时间限制，只训练了 50 多轮，考虑使用 cuda 进行 gpu 加速或许可以提升训练速度。

Generated Samples (Diffusion)



尽管训练的轮次不足,扩散模型生成的图片仍然具有很不错的清晰度,相较于 VAE 和 GAN 的结果都更加清晰, 如果训练轮次足够应该能有非常好的效果。

任务四 强化学习

4.1. 任务说明

本章任务聚焦于深度强化学习（DRL）在连续状态空间下的离散动作控制控制。实验基于 OpenAI 的 CartPole（车杆平衡）经典仿真物理环境，智能体需要根据环境反馈的 4 维状态向量（包含小车位置、小车速度、车杆角度、车杆角速度），在倾角不超过 15° 且位移不超过 2.4 个单位的边界约束下，通过双位离散动作（左移/右移）实施高频控制。

车杆游戏，OpenAI构建的简单强化学习仿真环境，游戏里面有一个小车，上有竖着一根杆子，每次重置后的初始状态会有所不同。小车需要左右移动来保持杆子竖直，需要满足以下两个条件：1. 杆子倾斜的角度 θ 不能大于 15° 2. 小车移动的位置 x 需保持在一定范围（中间到两边各2.4个单位长度）



本实验旨在通过搭建深度 Q 网络（DQN），实现时序差分（TD）目标值的自适应拟合与贝尔曼方程的收敛迭代，打通‘环境感知-网络推理-动作执行-奖励回传-参数调优’的闭环具身智能控制链路。

4.2. 任务 构建 DQN 网络进行车杆平衡控制

4.2.1. DQN 介绍

● 强化学习的基本要素

要理解深度强化学习，首先需要掌握其基础框架中的几个核心角色和它们之间的关系。

智能体 (Agent)

角色：学习者与决策者。

职责：观察环境状态，根据所学策略选择动作，执行动作，并从环境中接收反馈

（新状态和奖励）。

环境 (Environment)

角色：智能体交互的一切外部事物。

职责：接收智能体的动作，更新自身状态，并给出相应的奖励。

状态 (State, s)

定义：环境在某一时刻的具体情况描述。在深度强化学习中，状态通常是高维的，比如一帧游戏图像。

动作 (Action, a)

定义：智能体在给定状态下可以做出的选择。例如，在游戏中可能是"向上"、"向左"、"开火"等。

奖励 (Reward, r)

定义：环境对智能体动作的即时反馈信号，是一个标量值。奖励是智能体学习的"指南针"，其目标是最大化长期累积奖励。

策略 (Policy, π)

定义：智能体的行为准则，是从状态到动作的映射函数。它告诉智能体在什么状态下应该做什么动作。策略可以是确定性的 ($a = \pi(s)$)，也可以是随机性的 ($a \sim \pi(a|s)$)。

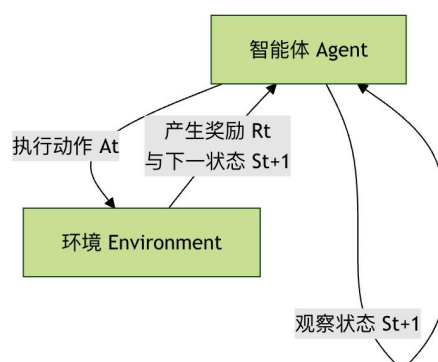
价值函数 (Value Function)

定义：用于评估状态或状态-动作对的好坏。它代表了从当前状态（或执行当前动作后）开始，未来能获得的预期累积奖励。

状态价值函数 $V(s)$ ：在状态 s 下，遵循当前策略所能获得的预期回报。

动作价值函数 $Q(s, a)$ ：在状态 s 下执行动作 a ，然后遵循当前策略所能获得的预期回报。

智能体与环境的交互是一个持续的循环过程，可以用以下流程图清晰地表示：



这个循环不断重复，智能体收集大量的交互数据 (s, a, r, s') ，并利用这些数据来改进自己的策略。

- 基于价值的深度 Q 网络

这类算法的核心是学习最优的动作价值函数 $Q(s, a)$ 。一旦学到了准确的 Q 函数，最优策略就很简单：在每个状态 s 下，选择能使 $Q(s, a)$ 最大的动作 a 。

深度 Q 网络 (DQN) 是里程碑式的工作。它用深度神经网络来近似复杂的 Q 函数。DQN 的关键技术创新在于：

经验回放 (Experience Replay): 智能体将交互经验 (s, a, r, s') 存储在一个记忆库中。训练时，随机从库中抽取一批经验进行学习。这打破了数据间的相关性，使训练更稳定、更高效。

目标网络 (Target Network): 使用一个结构相同但参数更新较慢的"目标网络"来计算学习目标 (Q 目标值)，而用另一个"在线网络"进行动作选择和实时更新。这解决了训练中目标值不断移动的问题，大大提高了稳定性。

一个简化的 DQN 训练流程：

1. 初始化在线网络 Q 和目标网络 Q_{target} （参数相同），清空经验回放池。
2. 智能体根据当前状态 s ，以一定概率随机或根据 Q 网络选择动作 a 。
3. 执行动作，环境返回奖励 r 和新状态 s' ，将经验 (s, a, r, s') 存入回放池。
4. 从回放池中随机采样一批经验。
5. 对于每个样本，计算目标 Q 值： $y = r + \gamma * \max_{a'} Q_{\text{target}}(s', a')$ 。其中 γ 是折扣因子，用于权衡即时奖励和未来奖励。
6. 以 $(y - Q(s, a))^2$ 作为损失，通过梯度下降更新在线网络 Q 的参数。
7. 每隔一定步数，将在线网络的参数复制给目标网络。
8. 重复步骤 2-7。

优点与局限：

优点：样本效率相对较高，训练相对稳定。

局限：天然难以处理连续动作空间（因为需要计算 $\max_a Q(s, a)$ ），且通常只能学习确定性策略。

4.2.2. 关键代码及说明

(1) 环境与依赖

```
1. 环境与依赖

如未安装 Gym/Gymnasium，请先安装对应包。

import random
from collections import deque
import os
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

import gymnasium as gym

import matplotlib.pyplot as plt
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" # 预防绘图崩溃

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

✓ 0.0s
device: cpu
```

(2) 创建环境与网络

2. 创建环境

```
def reset_env(env, seed=None):
    try:
        obs, _ = env.reset(seed=seed)
    except TypeError:
        if seed is not None:
            env.seed(seed)
        obs = env.reset()
    return obs

def step_env(env, action):
    out = env.step(action)
    if len(out) == 5:
        obs, reward, terminated, truncated, _ = out
        done = terminated or truncated
    else:
        obs, reward, done, _ = out
    return obs, reward, done

env = gym.make("CartPole-v1")
obs = reset_env(env, seed=seed)
print("obs shape:", np.array(obs).shape, "action space:", env.action_space)

✓ 0.0s
obs shape: (4,) action space: Discrete(2)
```

3. DQN 网络（单网络版本）

```
class QNetwork(nn.Module):
    def __init__(self, obs_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, action_dim),
        )

    def forward(self, x):
        return self.net(x)

✓ 0.0s
```

4. 训练配置

```
gamma = 0.99
learning_rate = 10e-4
max_episodes = 800
max_steps = 500

✓ 0.0s
```

(3) 训练 DQN

```
obs_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

q_net = QNetwork(obs_dim, action_dim).to(device)
optimizer = optim.Adam(q_net.parameters(), lr=learning_rate)

def train_step(state, action, reward, next_state, done):
    state_t = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    next_state_t = torch.tensor(next_state, dtype=torch.float32, device=device).unsqueeze(0)
    q_next = q_net(next_state_t).max(1)[0].item()
    target = reward + gamma * q_next * (1.0 - done)
    q_value = q_net(state_t)[0, action]
    target_t = torch.tensor(target, dtype=torch.float32, device=device)
    loss = F.mse_loss(q_value, target_t)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()

rewards = []
losses = []
print("开始训练 DQN...")

for episode in range(1, max_episodes + 1):
    state = reset_env(env, seed=seed + episode)
    episode_reward = 0.0
    eps = 0.2 / (episode / 5 + 1)

    for _ in range(max_steps):
        if random.random() < eps:
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                q_values = q_net(torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0))
                action = int(torch.argmax(q_values, dim=1).item())

        next_state, reward, done = step_env(env, action)
        loss = train_step(state, action, reward, next_state, float(done))
        losses.append(loss)
        state = next_state
        episode_reward += reward

        if done:
            break

    rewards.append(episode_reward)
    if episode % 10 == 0:
        avg_r = np.mean(rewards[-10:])
        print(f"Episode {episode:03d} | Avg Reward(10): {avg_r:.2f} | Eps: {eps:.3f}")

✓ 3m 52.0s
```

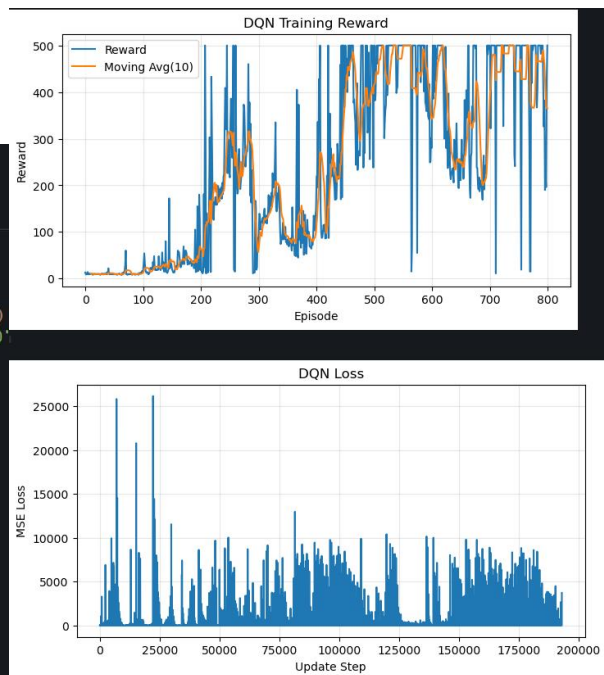
(4) 训练过程可视化

6. 训练过程可视化

```
plt.figure(figsize=(8, 4))
plt.plot(rewards, label="Reward")
if len(rewards) >= 10:
    ma = np.convolve(rewards, np.ones(10) / 10, mode="valid")
    plt.plot(range(9, 9 + len(ma)), ma, label="Moving Avg(10)")
plt.title("DQN Training Reward")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

if len(losses) > 0:
    plt.figure(figsize=(8, 4))
    plt.plot(losses)
    plt.title("DQN Loss")
    plt.xlabel("Update Step")
    plt.ylabel("MSE Loss")
    plt.grid(True, alpha=0.3)
    plt.show()

✓ 0.1s
```



设计总结与心得

通过本学期四个大任务的深度迭代实践，我完成了从基础前馈网络到复杂具身控制、从确定性判别分析到随机性概率生成的深度学习全栈知识体系的构建：

手写数字识别的经典案例（任务一）：从古老的多层感知机到最新的 transformer，通过实践我建立了对各个架构的认识，学习了梯度下降的原理，通过实践体会了学习率，网络架构深度等因素对模型训练的影响，对大模型有了更深入的认识。

数据异构性的拓宽（任务二）：图神经网络（GCN）的实践打破了传统欧几里得网格数据的限制，使我掌握了谱域层级拓扑特征聚合的机理；而 UNet 的对称结构与跳跃连接（Skip Connection）则向我展示了高层抽象语义与底层密集空间细节如何完美融合以实现像素级精确分割。

从判别到生成的范式转换（任务三）：在 VAE、GAN 及扩散模型的协同对比中，我学习到了三种方法各自的原理，通过实践也更加深刻的认识到了各自的优缺点，尤其是 GAN 困难的训练过程使我收益良多，对于参数和网络架构的选择和调整有了更深的理解。

强化学习与具身智能（任务四）：DQN 算法通过将深度网络作为非线性 Q 表拟合器，打通了具身智能在连续物理环境下的决策控制闭环，让我对强化学习有了初步认识。
